算法机考开卷手册

Python · Hot 100 · 按官方分类 · 17 类 / 共 100 题，已录入 100 · 2026-11-11

目录

0 输入输出模板

输入模板（14 条）
输出模板（8 条）

1 哈希 3/3

1.1 两数之和
1.2 字母异位词分组
1.3 最长连续序列

2 双指针 4/4

2.1 移动零
2.2 盛最多水的容器
2.3 三数之和
2.4 接雨水

3 滑动窗口 2/2

3.1 无重复字符的最长子串
3.2 找到字符串中所有字母异位词

4 子串 3/3

4.1 和为 K 的子数组
4.2 滑动窗口最大值
4.3 最小覆盖子串

5 普通数组 5/5

5.1 最大子数组和
5.2 合并区间
5.3 轮转数组
5.4 除自身以外数组的乘积
5.5 缺失的第一个正数

6 矩阵 4/4

6.1 矩阵置零
6.2 螺旋矩阵
6.3 旋转图像
6.4 搜索二维矩阵 II

7 链表 14/14

7.1 相交链表

7.2 反转链表
7.3 回文链表
7.4 环形链表
7.5 环形链表 II
7.6 合并两个有序链表
7.7 两数相加
7.8 删除链表的倒数第 N 个结点
7.9 两两交换链表中的节点
7.10 K 个一组翻转链表
7.11 随机链表的复制
7.12 排序链表
7.13 合并 K 个升序链表
7.14 LRU 缓存

8 二叉树 15/15

8.1 二叉树的中序遍历
8.2 二叉树的最大深度
8.3 翻转二叉树
8.4 对称二叉树
8.5 二叉树的直径
8.6 二叉树的层序遍历
8.7 将有序数组转换为二叉搜索树
8.8 验证二叉搜索树
8.9 二叉搜索树中第 K 小的元素
8.10 二叉树的右视图
8.11 二叉树展开为链表
8.12 从前序与中序遍历序列构造二叉树
8.13 路径总和 III
8.14 二叉树的最近公共祖先
8.15 二叉树中的最大路径和

9 图论 4/4

9.1 岛屿数量
9.2 腐烂的橘子
9.3 课程表
9.4 实现 Trie (前缀树)

10 回溯 8/8

10.1 全排列
10.2 子集
10.3 电话号码的字母组合
10.4 组合总和
10.5 括号生成
10.6 单词搜索

10.7 分割回文串
10.8 N 皇后

11 二分查找 6/6

11.1 二分查找（闭区间模板）
11.2 搜索插入位置
11.3 搜索二维矩阵
11.4 在排序数组中查找元素的第一个和最后一个位置
11.5 搜索旋转排序数组
11.6 寻找旋转排序数组中的最小值
11.7 寻找两个正序数组的中位数

12 栈 5/5

12.1 有效的括号
12.2 最小栈
12.3 字符串解码
12.4 每日温度
12.5 柱状图中最大的矩形

13 堆 3/3

13.1 数组中的第 k 个最大元素
13.2 前 K 个高频元素
13.3 数据流的中位数

14 贪心算法 4/4

14.1 买卖股票的最佳时机
14.2 跳跃游戏
14.3 跳跃游戏 II
14.4 划分字母区间

15 动态规划 10/10

15.1 爬楼梯
15.2 杨辉三角
15.3 打家劫舍
15.4 完全平方数
15.5 零钱兑换
15.6 单词拆分
15.7 最长递增子序列
15.8 乘积最大子数组
15.9 分割等和子集
15.10 最长有效括号

16 多维动态规划 5/5

16.1 不同路径
16.2 最小路径和

16.3 最长回文子串
16.4 最长公共子序列
16.5 编辑距离

17 技巧 5/5

17.1 只出现一次的数字
17.2 多数元素
17.3 颜色分类
17.4 下一个排列
17.5 寻找重复数

0 ACM 输入输出模板

左边是样例，右边是对应代码；下方给出变量结果或输出所需的变量。每张卡片的其他格式单独标 import、样例和结果，按题面替换，不与主代码一起执行。函数判题提交指定类/函数；ACM 模式自行读取、调用算法并输出。示例 solve() 需按题意实现并返回答案，不能只定义 class Solution 而不调用。

输入模板

14 张卡片，另有 8 个变体

先看题面按行还是按 token 分隔。整行字符串保留空白；数字、单词可按 token 取。空数组是否占行按题面，不能统一多读或少读一行。卡片内的其他格式按题面选用，不与主代码同时执行。

① 一行两个整数

不用 import

输入样例 · 1 行

```
3 5
```

```
a, b = map(int, input().split())
```

→ a = 3, b = 5 | split() 不带参数，按任意空白切，多个空格、Tab 都能吃。

② 先给 n，下一行给 n 个数

不用 import

输入样例 · 2 行

```
5
1 2 3 4 5
```

```
n = int(input())
nums = list(map(int, input().split()))
```

→ n = 5, nums = [1, 2, 3, 4, 5] | 下一行恰好 n 个数时用主代码。n = 0 时，只有题面仍给一空行才读该行；若省略数组行，或字段任意折行，用③。

其他格式 · n 和数组在同一行

不用 import

输入样例 · 1 行

```
3 1 2 3
```

```
n, *nums = map(int, input().split())
```

→ n = 3, nums = [1, 2, 3] | 这一行必须恰有 n + 1 个整数；输入只有 0 时得到空数组，不能把后续字段也放进这一行。

其他格式 · 完整 ACM 示例：读入、调用函数、输出

不用 import

输入样例 · 2 行

```
3
4 -1 2
```

```
def solve(nums):
    return sum(nums)

n = int(input())
nums = list(map(int, input().split()))
print(solve(nums))
```

→ 标准输出为 5，末尾换行。 | 本例规定第二行恰有 n 个整数。套用手册里的类方法时，先复制题解，再把 solve(nums) 换成 Solution().方法名(nums)，参数和输出格式按题面调整。

③ 按个数读 n 个数（允许折行）

import sys

输入样例 · 3 行

```
5
1 2 3
4 5
```

```
data = (x for line in sys.stdin for x in line.split
n = int(next(data))
nums = [int(next(data)) for _ in range(n)]
```

→ `n = 5, nums = [1, 2, 3, 4, 5]` | 逐行生成 token，不一次读完。n 可独占一行，也可与数组同一行；n = 0 不取数组元素。后续字段继续 next(data)，例如再读一个整数可写 k = int(next(data))。不要混用 input()；空白有意义的字符串不能这样读。

④ n 行，每行一个

不用 import

输入样例 · 4 行

```
3
10
20
30
```

```
n = int(input())
nums = [int(input()) for _ in range(n)]
```

→ `n = 3, nums = [10, 20, 30]` | 整数也能用 read().split() 一次读取。若每行是字符串，把 int(input()) 整体换成 input()；只有题面允许忽略首尾空白时才加 .strip()。

⑤ 逗号整数 / JSON 数组

不用 import

输入样例 · 1 行

```
1,2,3
```

```
s = input().strip()
nums = list(map(int, s.split(','))) if s else []
```

→ `nums = [1, 2, 3]` | 主代码约定空行表示空数组；否则输入为无括号、无空字段的逗号整数。int() 接受两侧空格。1,,2 或末尾逗号仍会报错，不擅自删空项。带括号看下面的 JSON 写法。

其他格式 · 带括号的 JSON 数组

import json

输入样例 · 1 行

```
[1, true, null, "中"]
```

```
nums = json.loads(input())
```

→ `nums = [1, True, None, '中']` | 按标准 JSON 写：字符串用双引号，布尔值和空值写 true / false / null；单引号、True、None、nums = [...] 都不是此格式。多行单一 JSON 文档可用 json.load(sys.stdin)，需另 import sys。

⑥ 浮点数

不用 import

输入样例 · 1 行

```
2.5 3.25
```

```
x, y = map(float, input().split())
```

→ `x = 2.5, y = 3.25` | 部分浮点运算有表示误差，如 0.1 + 0.2。print 不会额外制造误差；输出精度和小数位数按题面设置，不统一保留两位。

⑦ 第一行 T 组数据

不用 import

输入样例 · 3 行

```
2
3 5
4 1
```

```
T = int(input())
for _ in range(T):
    a, b = map(int, input().split())
    print(a + b)
```

→ 屏幕依次打出 8、5 | 主代码每组一行、恰好两个整数。变长数组用下面的 token 写法；示例用求和代替实际算法。

其他格式 · T 组，每组先给 n 再给 n 个数

import sys

输入样例 · 4 行

```
3
3 1 2 3
0
2 -1 4
```

```
data = (x for line in sys.stdin for x in line.split
T = int(next(data))
for _ in range(T):
    n = int(next(data))
    nums = [int(next(data)) for _ in range(n)]
    print(sum(nums))
```

→ 屏幕依次输出 6、0、3 | 字段可任意折行；n = 0 时没有数组 token。T = 0 不输出；一组已开始却缺少数据属于不完整输入。

⑧ 多组数据，读到文件尾（EOF）

import sys

输入样例 · 3 行

```
3 5
4 1
4 4
```

```
for line in sys.stdin:
    if not line.strip():
        continue
    a, b = map(int, line.split())
    print(a + b)
```

→ 屏幕依次打出 8、5、8 | 主代码每个非空行恰好两个整数；空行跳过，EOF 自动结束。每组含变长数组时用下面的按数量读取。

其他格式 · 无 T，每组先给 n，读到 EOF

import sys

输入样例 · 3 行

```
3 1 2 3
0
2 -1 4
```

```
data = (x for line in sys.stdin for x in line.split
for first in data:
    n = int(first)
    nums = [int(next(data)) for _ in range(n)]
    print(sum(nums))
```

→ 屏幕依次输出 6、0、3 | for 每次取得下一组的 n，组内继续从同一迭代器取数。n = 0 输出 0；空输入不输出；不能忽略一组中途缺失的数。

⑨ 读到哨兵值结束

import sys

输入样例 · 3 行

```
3 5
4 1
0 0
```

```
for line in sys.stdin:
    if not line.strip():
        continue
    a, b = map(int, line.split())
    if a == 0 and b == 0:
        break
    print(a + b)
```

→ 屏幕打出 8、5，读到 0 0 就停 | 每个非空行恰好两个整数，遇 0 0 或 EOF 都结束。若终止行只有 -1 或 0，须在解包前判断 line.strip()，不能先读成两个数。

⑩ 矩阵 / 网格

不用 import

输入样例 · 3 行

```
2 3
1 2 3
4 5 6
```

```
n, m = map(int, input().split())
g = [list(map(int, input().split())) for _ in range
```

→ `g = [[1, 2, 3], [4, 5, 6]]`, `g[1][2] = 6` | 主代码每行 m 个空白分隔整数。下方是边表转邻接表；无空格字符网格看⑪。

其他格式 · 图：边表转邻接表

不用 import

输入样例 · 3 行

```
3 2
1 2
2 3
```

```
n, m = map(int, input().split())
g = [[] for _ in range(n)]
for _ in range(m):
    u, v = map(int, input().split())
    u -= 1
    v -= 1
    g[u].append(v)
    g[v].append(u)
```

→ `g = [[1], [0, 2], [1]]` | 此例无向、无权，输入编号 1..n，内部改为 0..n-1。有向图不加反向边；带权图读 u, v, w，再存 (v, w)。不要写 [[]] * n。

---

⑪ 字符网格（首行 n m）

不用 import

输入样例 · 3 行

```
2 3
.#.
#..
```

```
n, m = map(int, input().split())
g = [list(input()) for _ in range(n)]
```

→ `g = [['.', '#', '.'], ['#', '.', '.']]` | 每行恰好 m 个字符，首尾空格若为有效格子也保留。若首行只有 n，只把第一行改成 n = int(input())。input() 去掉末尾的换行符。若输入可能是 CRLF，逐行去掉行尾后再转 list，参见⑫。

---

⑫ 整行字符串（空格要保留）

不用 import

输入样例 · 1 行

```
hello world
```

```
s = input()                # 原样整行
```

→ `s = 'hello world'` | input() 去掉末尾的换行符，保留其他空白；遇 EOF 抛 EOFError。若输入可能是 CRLF，使用下方的去行尾写法。

其他格式 · 读到 EOF，保留空行和首尾空格

import sys

输入样例 · 3 行

```
 a

b
```

```
lines = []
for line in sys.stdin:
    if line.endswith('\r\n'):
        line = line[:-2]
    elif line.endswith('\n'):
        line = line[:-1]
    lines.append(line)
```

→ `lines = [' a ', '', 'b']` | for 读到 EOF 自动结束，空行仍会读到。代码只去掉 LF 或 CRLF，保留其他空白，最后一行没有换行也能读。若改用 readline()，返回 '' 才是 EOF。

---

⑬ 整行字符串并去掉首尾空白

不用 import

输入样例 · 1 行

```
hello world
```

```
s = input().strip()       # 两端空白全去
```

→ s = 'hello world' | strip() 去掉首尾空白，保留中间空格；题目要求原样保留时用上一条。纯空白行判空可写 if not s.strip():。

⑭ 批量读取（数值 / 单词）

import sys

输入样例 · 2 行

```
5
1 2 3 4 5
```

```
data = iter(sys.stdin.buffer.read().split())
n = int(next(data))
nums = [int(next(data)) for _ in range(n)]
```

→ n = 5, nums = [1, 2, 3, 4, 5] | 主代码按 token 一次读完，需额外内存；数据量大不一定应全读，可用③逐行生成 token。后续继续 next(data)，不要混用 input()。整行空白有意义、或交互题不能这样读。

其他格式 · 批量读取 UTF-8 单词

import sys

输入样例 · 1 行

```
2 hello 世界
```

```
data = iter(sys.stdin.buffer.read().split())
n = int(next(data))
words = [next(data).decode('utf-8') for _ in range(
```

→ words = ['hello', '世界'] | buffer 读出 bytes，split() 按 ASCII 空白分词。int() 可直接转数值，文本要 decode。这里约定 UTF-8；其他编码按题面。str(b'abc') 不是解码；字段内空格不能靠 split 保留。

输出模板

8 张卡片，另有 3 个变体

输出格式逐字按题面：空格、换行、大小写、括号和精度都属于答案。空结果是否输出空行按题面；卡片给定变量需先由算法算出。

① 一个值

不用 import

屏幕输出 · 1 行

```
8
```

```
print(ans)
```

给定 ans = 8 | print(ans) 适合标量结果；一维空格分隔看②，矩阵看③/⑥。题面要求 JSON 时用下面的写法，不能直接输出 Python 容器表示。

其他格式 · 题面要求 JSON 格式的结果

import json

屏幕输出 · 1 行

```
[["中"], true, null]
```

```
s = json.dumps(ans, ensure_ascii=False, allow_nan=F
print(s)
```

给定 ans = [['中'], True, None] | JSON 使用双引号、true / false / null。若要求紧凑格式，设 separators=(',', ':')；allow_nan=False 会拒绝 NaN 和无穷值，输出保持标准 JSON。普通空格数组或矩阵不使用此格式。

② 一行空格分隔

不用 import

屏幕输出 · 1 行

```
1 2 3
```

```
print(*nums)                # 空格分隔
# print(*nums, sep=', ')   # 逗号分隔：替换上一行
# print(*nums, sep='', end='')   # 拼接且不换行
```

给定 nums = [1, 2, 3] | 备选写法只选一种；空列表用主代码会输出空行。end 控制末尾字符，sep 控制元素之间的分隔符。交互题按协议用 flush=True；正式提交不输出提示或调试信息。

③ 每行一个

不用 import

屏幕输出 · 3 行

```
1
2
3
```

```
for x in nums:
    print(x)
```

`给定 nums = [1, 2, 3]` | 空列表不输出。非空时也可写 print(*nums, sep='\n')；该写法对空列表会多输出一个空行。

其他格式 · 整数矩阵逐行输出

不用 import

屏幕输出 · 2 行

```
1 2
3 4
```

```
for row in g:
    print(*row)
```

`给定 g = [[1, 2], [3, 4]]` | 每行空格分隔，行末不加多余空格；g 为空时不输出。空行是否为合法矩阵行按题面。

---

④ 保留小数

不用 import

屏幕输出 · 1 行

```
2.500000
```

```
print(f'{ans:.6f}')        # 小数点后 6 位
```

`给定 ans = 2.5` | 示例按要求保留 6 位；位数按题面调整。固定小数位用 f-string；round() 返回数值，不保证显示末尾的 0。格式化不能修复此前的计算误差。

---

⑤ 布尔值（True / true / YES）

不用 import

屏幕输出 · 1 行

```
True
```

```
print(ans)                              # True / False
# print('true' if ans else 'false')  # 要小写就换这句
# print('YES' if ans else 'NO')      # 判定题常用
```

`给定 ans = True` | print(True) 打出首字母大写的 True；题面要 true 或 YES 就注释掉第一行、放开对应那行，大小写不一致等于答案错。

---

⑥ 元素不是 int，拼成一行

不用 import

屏幕输出 · 1 行

```
a b c
```

```
print(' '.join(words))              # 全是 str
# print(' '.join(map(str, nums)))  # 有 int 先转
```

`给定 words = ['a', 'b', 'c']; nums = [1, 2, 3]` | join 只吃 str，混进 int 直接 TypeError；两行挑一行用。

其他格式 · 字符矩阵逐行输出

不用 import

屏幕输出 · 2 行

```
.#.
#..
```

```
for row in g:
    print(''.join(row))
```

`给定 g = [['.', '#', '.'], ['#', '.', '.']]` | 行内不插空格，每个元素必须是字符串。不要用 print(*row)，它默认插空格；g 为空时不输出。

---

⑦ 多组结果按行输出

不用 import

屏幕输出 · 2 行

```
8
5
```

```
out = []
for _ in range(T):
    out.append(str(solve()))
if out:
    print('\n'.join(out))
```

给定 T = 2, solve() 依次返回 8、5 | solve() 需按题意实现，每次返回一个标量或已经按题意拼好的单行字符串；先转字符串再 join。T = 0 时不输出；批量输出额外占用内存。

⑧ 带 Case 编号

不用 import

屏幕输出 · 2 行

```
Case #1: 8
Case #2: 5
```

```
for i in range(1, T + 1):
    print(f'Case #{i}: {solve()}')
```

给定 T = 2, solve() 依次返回 8、5 | solve() 需按题意实现；Case 编号从 1 开始，井号、冒号和空格严格按题面要求。

1 哈希 3/3

1.1 两数之和 LC 1

在 nums 里找两个下标，使对应的数相加等于 target，返回这两个下标（答案唯一，同一个元素不能用两次）。

例 nums = [2,7,11,15], target = 9 → [1,0]（下标 0 的 2 加下标 1 的 7；两个下标谁先谁后都算对）

```
class Solution:
    def twoSum(self, nums, target):
        hash = {}                           # 键 = 见过的数字，值 = 它的下标
        for x in range(len(nums)):
            res = target - nums[x]          # 还差多少才凑成 target
            if res in hash:
                return [x, hash[res]]       # 差的那个见过，两个下标直接交
            else:
                hash[nums[x]] = x           # 没见过，把当前数字和坐标存进去
```

1.2 字母异位词分组 LC 49

把 strs 里互为字母异位词的串分到同一组，返回所有组（组内顺序、组间顺序都随意）。

例 strs = ["eat","tea","tan","ate","nat","bat"] → [["eat","tea","ate"],["tan","nat"],["bat"]]

```
from collections import defaultdict

class Solution:
    def groupAnagrams(self, strs):
        d = defaultdict(list)               # 键 = 排好序的串，值 = 这一组的原串
        for i in strs:
            _ = "".join(sorted(i))          # 异位词排完序一模一样，拿它当组名
            d[_].append(i)                  # 组不存在时自动建空 list，直接塞
        return list(d.values())             # 只要各组，不要键
```

注 不用 defaultdict 也行：先 if _ not in d: d[_] = [] 再 d[_].append(i)，多写两行；defaultdict(list) 只是把「缺键先建空 list」这一步替你做了。

1.3 最长连续序列 LC 128

在 nums 里找最长的一段连续整数（值连续、位置不必相邻，重复的算一个），返回这段的长度；题目要求 O(n)，所以不能排序。

例 nums = [100,4,200,1,3,2] → 4（1,2,3,4 连成一串）

```
class Solution:
    def longestConsecutive(self, nums):
        if not nums:                    # 空数组直接 0，否则下面 max 会炸
            return 0
        nums = set(nums)                # 去重，成员查询平均 O(1)
        hash = {}                       # 键 = 一段连续区的起点，值 = 这段多长
        for i in nums:
            if i - 1 in nums:           # 前面还有数，说明 i 不是起点，跳过
                continue
            a = 1                       # 从 i 开始往右数，先算它自己
            b = i
            while b + 1 in nums:
                b += 1
                a += 1
            hash[i] = a                 # 这一段以 i 打头，长度记成 a
        return max(list(hash.values()))
```

注 `max(list(hash.values()))` 里的 `list()` 可以省：`max` 直接吃 `dict_values` 也行，写成 `max(hash.values())` 少一次拷贝，结果一样。

专题小结

① 两数之和：一边走一边把见过的数记在本子上，每到一个新数就翻本子看「还差的那个」在不在，在就直接交答案——不用拿每个数去跟后面所有数试一遍。
② 字母异位词分组：互为异位词的串，把字母排个序就长得一模一样，于是拿「排完序的串」当抽屉标签，原串往对应抽屉里一扔，最后把抽屉挨个倒出来就是答案。
③ 最长连续序列：从头挨个往后数的话，同一段会被翻来覆去数很多遍；只从「前面没有邻居」的数开始数，每段就只数一次，数完把段长记下来取最大。

核心 共同点是用哈希表或集合加快重复查询：两数之和按补数查下标，异位词按排序后的字符串分组，最长连续序列只从段首扩展。哈希查询通常为平均 O(1)，不是所有处理都为 O(n)：异位词分组仍需逐串排序；最长连续序列虽有内层 while，但每个不同整数只属于一段，因此总计平均 O(n)。

2 双指针

2.1 移动零 LC 283

把 nums 中的所有 0 移到末尾，同时保持非零元素原有顺序；直接修改 nums，不返回数组。

例 nums = [0,1,0,3,12] → nums 变为 [1,3,12,0,0]（返回值是 None）

```
class Solution:
    def moveZeroes(self, nums):
        l = 0                                   # 下一个非零元素应放的位置
        r = 0                                   # 当前扫描的位置
        n = len(nums)
        while r < n:
            if nums[r] != 0:                    # 遇到非零数，放进前面的有效区
                nums[l], nums[r] = nums[r], nums[l]  # 原地交换，保留非零数先后顺序
                l += 1                          # 非零区扩大一位
            r += 1                              # 不论是否为零，都继续向右扫描
```

2.2 盛最多水的容器 LC 11

给定非负整数数组 height，每个下标对应一条竖线。选两条线和横轴构成容器，返回能盛的最大水量。容器不能倾斜。

例 height = [1,8,6,2,5,4,8,3,7] → 49（下标 1 和 8）；height = [1,1] → 1

```
class Solution:
    def maxArea(self, height):
        n = len(height)
        l = 0
        r = n - 1
        ans = 0
        while l < r:
            area = min(height[r], height[l]) * (r - l)
            ans = max(area, ans)
            if height[l] <= height[r]:
                l += 1                      # 左侧较短或等高，舍弃左端
            else:
                r -= 1                      # 右侧较短，舍弃右端
        return ans
```

注 面积是两端较短的高度乘宽度。若左端较短，固定左端再把右端往里移，宽度会变小，高度也不会超过原左端，所以这些组合都不可能更好，可以直接舍弃左端。
右端较短时同理。两端等高时任选一侧移动，原稿选择移动左端。时间 O(n)，额外空间 O(1)。

2.3 三数之和 LC 15

给定整数数组 nums，返回由三个不同下标组成、元素和为 0 的所有三元组。按值去重，结果顺序不限。

例 nums = [-1,0,1,2,-1,-4] → [[-1,-1,2],[-1,0,1]]；nums = [0,0,0,0] → [[0,0,0]]

```
class Solution:
    def threeSum(self, nums):
        n = len(nums)
        nums.sort()
        ans = []
        for i in range(n):
            if nums[i] > 0:
                break                       # 后面的数都为正，三数和不可能为 0
            if i > 0 and nums[i-1] == nums[i]:
                continue                    # 同一个首项只处理一次
            l = i + 1
            r = n - 1
            while l < r:
                s = nums[i] + nums[l] + nums[r]
                if s < 0:
                    l += 1                  # 和偏小，换更大的左值
                elif s > 0:
                    r -= 1                  # 和偏大，换更小的右值
                else:
                    ans.append([nums[i], nums[l], nums[r]])
                    r -= 1
                    l += 1
                    while l < r and nums[l-1] == nums[l]:
                        l += 1              # 跳过刚用过的左值
                    while l < r and nums[r+1] == nums[r]:
                        r -= 1              # 跳过刚用过的右值
        return ans
```

注 排序后固定 nums[i]，只在它右边用双指针寻找另两个数。和偏小就增大左值，和偏大就减小右值。两层都要去重。外层跳过重复首项，内层命中后把两端都移动，再跳过相同值。sort() 会修改输入数组。总时间 O(n²)。双指针扫描只用 O(1) 额外空间，Python 的 sort() 可能使用 O(n) 辅助空间，答案另计。

2.4 接雨水 LC 42

给定非负整数数组 height，每个元素是一根宽度为 1 的柱子高度。返回下雨后能接住的雨水总量。

例 height = [0,1,0,2,1,0,1,3,2,1,2,1] → 6；height = [2,0,2] → 2；height = [1,2,3] → 0

```
class Solution:
    def trap(self, height):
        n = len(height)
        l = 0
        r = n - 1
        ans = 0
        max_l = 0
        max_r = 0
        while l <= r:
            if max_l < max_r:
                if height[l] > max_l:
                    max_l = height[l]
                else:
                    ans += max_l - height[l]  # 左侧已见最高柱决定水位
                l += 1
            else:
                if height[r] > max_r:
                    max_r = height[r]
                else:
                    ans += max_r - height[r]  # 右侧已见最高柱决定水位
                r -= 1
        return ans
```

注 max_l 和 max_r 记录两边已经处理过的最高柱。左边的最高柱较矮时，右边已经有足够高的挡板，左指针这一格就能结算。右侧较矮或两边一样高时，按原稿结算右边。

例如左侧已见最高柱为 2，右侧为 4，左指针当前柱高为 1，那么这格接 2-1 = 1 单位水。若当前柱反而高过左侧最高柱，这格不接水，只更新左侧最高柱。

这里每轮结算一个位置，所以 l == r 时也要处理。和盛水容器不同，本题比较的是两侧已见最高值，并逐格累加。时间 O(n)，额外空间 O(1)。

专题小结

① 移动零　快指针扫数组，慢指针指向下一个非零元素的位置。

② 盛最多水的容器　先算当前面积，舍弃较矮一端。

③ 三数之和　排序后固定一个数，按和的大小移动两端，命中后跳过重复值。

④ 接雨水　比较两侧已见最高柱，先结算受较低水位限制的一侧。

核心 先想清楚移动指针时排除了哪些候选，或者确定了哪个位置的答案。容器比较当前两根柱，接雨水比较两侧已见最高柱。

3 滑动窗口

3.1 无重复字符的最长子串 LC 3

给定字符串 s，返回不含重复字符的最长连续子串的长度。

例 s = "abcabcbb" → 3；s = "abba" → 2；s = "" → 0

```
class Solution:
    def lengthOfLongestSubstring(self, s):
        n = len(s)
        l, r = -1, 0
        last = {}
        ans = 0
        while r < n:
            if s[r] not in last:
                last[s[r]] = r
                ans = max(ans, r - l)
            elif last[s[r]] > l:
                l = last[s[r]]          # 把窗口内的旧重复字符移到边界之外
                last[s[r]] = r
            elif last[s[r]] < l:
                last[s[r]] = r          # 旧位置在窗口之外，可以直接扩张
                ans = max(ans, r - l)
            r += 1
        return ans
```

注 窗口是 (l, r]，l 本身不在窗口里。字典记录字符最近出现的位置。遇到窗口内的重复字符时，左边界右移，这一轮的窗口长度不会超过上一轮，所以无需更新 ans。

原稿的 last[s[r]] == l 不会出现。每次把 l 改成某字符的旧位置，同一轮也把该字符的记录更新为 r，旧位置不会再留在字典里。时间 O(n)，额外空间 O(d)，d 是不同字符数。

3.2 找到字符串中所有字母异位词 LC 438

给定字符串 s 和非空字符串 p，返回 s 中所有与 p 字符次数完全相同的子串的起始下标。

例 `s = "cbaebabacd", p = "abc" → [0,6]; s = "baa", p = "aa" → [1]`

```
from collections import Counter

class Solution:
    def findAnagrams(self, s, p):
        cnt = Counter(p)
        ans = []
        l = 0
        for i in range(len(s)):
            cnt[s[i]] -= 1
            while cnt[s[i]] < 0:         # 右端字符超额，左端逐个退出
                cnt[s[l]] += 1
                l += 1
            if i - l + 1 == len(p):
                ans.append(l)
        return ans
```

注 cnt[c] 表示 p 还允许窗口放入几个 c。每次只有右端新加入的字符可能使余额变负。恢复非负后，所有字符都没超额，窗口长度恰好等于 len(p) 时，字符次数就一定相同。
左右端各走一遍。时间 O(len(s)+len(p))，额外空间为计数表大小，不计答案。

专题小结

最长无重复子串记录上次出现的位置，找到异位词记录每种字符的余额。

核心 先明确窗口里允许什么，再决定左端何时移动和答案何时记录。

4 子串

4.1 和为 K 的子数组 LC 560

给定非空整数数组 nums 和整数 k，返回和恰好等于 k 的非空连续子数组个数。

例 `nums = [1,1,1], k = 2 → 2; nums = [1,-1,0], k = 0 → 3`

```
from collections import defaultdict

class Solution:
    def subarraySum(self, nums, k):
        n = len(nums)
        pre = [0] * n
        pre[0] = nums[0]
        counts = defaultdict(int)
        counts[0] = 1                   # 数组之前有一个和为 0 的空前缀
        ans = 0
        for i in range(1, n):
            pre[i] = pre[i-1] + nums[i]
        for i in range(n):
            ans += counts[pre[i] - k]  # 只查当前下标之前的前缀
            counts[pre[i]] += 1
        return ans
```

注 pre[i] 包含 nums[i]。当前前缀减去某个旧前缀等于 k，就确定了一个以 i 结尾的子数组。空前缀可计入从下标 0 开始的答案。
先查询再登记，避免 k 为 0 时把当前前缀和自己相减算成空子数组。时间 O(n)，额外空间 O(n)。

4.2 滑动窗口最大值 LC 239

给定数组 nums 和窗口长度 k，返回窗口从左到右移动时每个窗口的最大值。

例 nums = [1,3,-1,-3,5,3,6,7], k = 3 → [3,3,5,5,6,7]；nums = [2,2,1], k = 2 → [2,2]

```
from collections import deque

class Solution:
    def maxSlidingWindow(self, nums, k):
        q = deque()
        n = len(nums)
        ans = [0] * (n - k + 1)
        for r in range(n):
            l = r - k + 1
            while q and q[0] == l - 1:
                q.popleft()             # 删除刚离开窗口的队首
            while q and nums[r] > nums[q[-1]]:
                q.pop()                 # 新值更大，又比队尾更晚过期
            q.append(r)
            if l >= 0:
                ans[l] = nums[q[0]]     # 满 k 个元素后才记录
        return ans
```

注 队列存下标，队首到队尾的值非递增。比如队尾有 4、2，新值是 3，就弹掉 2 并把 3 放到 4 后面。相同值都保留，较早的那个到期时从队首删除。

每轮左边界只向右走一格，所以本轮刚过期的下标是 l-1。每个下标至多进出队各一次。时间 O(n)，辅助空间 O(k)。

4.3 最小覆盖子串 LC 76

返回 s 中能覆盖 t 每种字符及其出现次数的最短连续子串。找不到时返回空字符串，题目保证 t 非空。

例 `s = "ADOBECODEBANC", t = "ABC" → "BANC"; s = "a", t = "aa" → ""`

```
from collections import Counter

class Solution:
    def minWindow(self, s, t):
        r, l = 0, 0
        n = len(s)
        start = 0
        le = 10**9
        cnt_base = Counter(t)
        cnt_now = Counter()
        target = len(cnt_base)
        valid = 0
        while r < n:
            if s[r] in cnt_base:
                cnt_now[s[r]] += 1
                if cnt_now[s[r]] == cnt_base[s[r]]:
                    valid += 1
            while valid == target:
                if s[l] in cnt_base:
                    cnt_now[s[l]] -= 1
                    if cnt_now[s[l]] < cnt_base[s[l]]:
                        valid -= 1
                if r - l + 1 <= le:
                    le = r - l + 1     # l 还未移动，记录的是退出前的窗口
                    start = l
                l += 1
            r += 1
        if le > n:
            return ""
        return s[start:start+le]
```

注 valid 统计达到要求的字符种类数。例如 t = "AA"，窗口内有两个 A 时才把 A 算为满足，第三个 A 不再增加 valid。
进入 while 时 [l, r] 已经覆盖 t。虽然先扣掉左端字符的计数，记录时用的仍是尚未移动的 l，所以记的是移除前的合法窗口。时间 O(len(s)+len(t))，计数表空间取决于 t 的字符种类，不计返回的子串。

专题小结

和为 K 用前缀差统计次数。滑动窗口最大值用单调队列保留还可能成为最大值的下标。最小覆盖用字符次数决定什么时候收缩。

核心 子数组和可以因负数变小，和为 K 用前缀表。窗口最大值和字符覆盖各有能随左右端更新的状态。

5 普通数组

5.1 最大子数组和 LC 53

给定非空整数数组 nums，返回所有非空连续子数组中的最大和。

例 nums = [−2,1,−3,4,−1,2,1,−5,4] → 6 ([4,−1,2,1]) ; nums = [−3,−1,−2] → −1

```
class Solution:
    def maxSubArray(self, nums):
        n = len(nums)
        prex = 0
        min_prex = 0
        ans = -10**9
        for i in range(n):
            prex += nums[i]
            min_prex = min(prex, min_prex)
            if prex == min_prex:
                ans = max(nums[i], ans)    # 当前差为 0 时不能把空段当答案
            else:
                ans = max(ans, prex - min_prex)
        return ans
```

注 用当前前缀和减去最小前缀和来取较大的区间和。原稿先更新最小前缀，若两者相等，这一轮就不能得到正的区间和，只拿当前单元素更新。
只要出现过正和，后面非正的区间就不会刷新最大值。若所有元素都非正，单元素分支会取到最大的那个数。时间 O(n)，额外空间 O(1)。

5.2 合并区间 LC 56

给定非空的闭区间数组，合并所有重叠区间，返回合并后的区间。端点相接也算重叠。

例 `intervals = [[1,3],[2,6],[8,10],[15,18]] → [[1,6],[8,10],[15,18]]; [[1,4],[4,5]] → [[1,5]]`

```
class Solution:
    def merge(self, intervals):
        intervals = sorted(intervals, key=lambda x: x[0])
        ans = []
        start, end = intervals[0]
        for i in range(1, len(intervals)):
            if intervals[i][0] > end:
                ans.append([start, end])    # 后面的起点更靠右，可以收尾
                start = intervals[i][0]
                end = intervals[i][1]
            else:
                end = max(end, intervals[i][1])
        ans.append([start, end])            # 收集最后一个区间
        return ans
```

注 按起点排序后，只需要维护当前区间的左右端点。遇到重叠时取较大的右端，避免把包含在里面的短区间当作新右端。时间 O(n log n)，排序和新列表额外空间 O(n)。

5.3 轮转数组 LC 189

给定非空数组 nums 和非负整数 k，把数组原地向右轮转 k 步，函数返回 None。

例 `nums = [1,2,3,4,5,6,7], k = 3 → nums 变为 [5,6,7,1,2,3,4]，返回 None`

```
class Solution:
    def rotate(self, nums, k):
        def reverse(x, i, j):
            while i < j:
                x[i], x[j] = x[j], x[i]
                i += 1
                j -= 1

        n = len(nums)
        k %= n
        reverse(nums, 0, n - 1)             # 整段反转，尾部 k 项来到前面
        reverse(nums, 0, k - 1)             # 恢复前 k 项的原相对顺序
        reverse(nums, k, n - 1)             # 恢复剩余项的原相对顺序
```

注 三次反转都直接交换原数组里的元素。k 先取模。k 为 0 时第一段为空，整段最终会反转回来。时间 O(n)，额外空间 O(1)。

5.4 除自身以外数组的乘积 LC 238

返回数组 ans，其中 ans[i] 等于 nums 除下标 i 外其余元素的乘积。要求 O(n) 时间，不能用除法。

例 `nums = [1,2,3,4] → [24,12,8,6]；nums = [-1,1,0,-3,3] → [0,0,9,0,0]`

```
class Solution:
    def productExceptSelf(self, nums):
        n = len(nums)
        pro = [1] * n
        ans = [1] * n
        pre = 1
        for i in range(n - 1, 0, -1):
            pro[i-1] = pro[i] * nums[i]    # i-1 右边所有数的乘积
        for i in range(n):
            ans[i] = pre * pro[i]
            pre *= nums[i]                 # 先算答案，再把当前位置算进前缀
        return ans
```

注 pro[i] 是严格位于 i 右边的乘积，pre 在计算 ans[i] 时是严格位于 i 左边的乘积。空的一边乘积是 1。时间 O(n)，不计结果数组，后缀表额外空间 O(n)。

5.5 缺失的第一个正数 LC 41

给定未排序的整数数组，返回数组里没出现过的最小正整数。要求 O(n) 时间和 O(1) 额外空间。

例 `nums = [3,4,-1,1] → 2；nums = [1,1] → 2；nums = [1,2,3] → 4`

```
class Solution:
    def firstMissingPositive(self, nums):
        n = len(nums)
        l = 0
        while l < n:
            x = nums[l]
            if x == l + 1 or x > n or x <= 0 or nums[x-1] == x:
                l += 1                     # 已归位、超范围，或目标位置已有同值
            else:
                nums[l], nums[x-1] = nums[x-1], nums[l]
        for i in range(n):
            if nums[i] != i + 1:
                return i + 1
        return n + 1
```

注 有效值 x 应该放到下标 x-1。短路判断先排除越界值，再访问 nums[x-1]。目标位已有同值时必须跳过，否则重复值会让交换停不下来。

每次实际交换至少固定一个新位置。扫描指针最多前进 n 次，交换也最多 n 次。时间 O(n)，额外空间 O(1)。代码会重新排列输入数组。

专题小结

最大子数组和看前缀差。合并区间排序后维护右端。轮转用三次反转。乘积用左右两侧的积。缺失正数把值放回对应下标。

核心 写数组题时先分清索引是位置还是值的目标位置。原地题要直接更新传入的对象。

6 矩阵

6.1 矩阵置零 LC 73

给定非空矩阵，只要原矩阵有一个 0，就把它所在行和列全部置 0。原地修改，返回 None。

例 matrix = [[1,1,1],[1,0,1],[1,1,1]] → matrix 变为 [[1,0,1],[0,0,0],[1,0,1]]，返回 None

```
class Solution:
    def setZeroes(self, matrix):
        m, n = len(matrix), len(matrix[0])
        row_zero = False
        line_zero = False
        for i in range(n):
            if matrix[0][i] == 0:
                row_zero = True
        for i in range(m):
            if matrix[i][0] == 0:
                line_zero = True

        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][j] == 0:
                    matrix[i][0] = 0    # 用首列标记行
                    matrix[0][j] = 0    # 用首行标记列
        for j in range(1, n):
            if matrix[0][j] == 0:
                for i in range(1, m):
                    matrix[i][j] = 0
        for i in range(1, m):
            if matrix[i][0] == 0:
                for j in range(1, n):
                    matrix[i][j] = 0

        if row_zero:
            for j in range(n):
                matrix[0][j] = 0
        if line_zero:
            for i in range(m):
                matrix[i][0] = 0
```

注 首行和首列要兼作标记，所以先记下它们自己是否原本就有 0，最后再处理它们。扫描和置零分开，避免把刚写入的 0 当作新的置零来源。时间 O(m × n)，额外空间 O(1)。

6.2 螺旋矩阵 LC 54

给定矩形矩阵，从左上角开始顺时针向内读取，返回访问到的元素列表。

例 `matrix = [[1,2,3],[4,5,6]] → [1,2,3,6,5,4]；matrix = [[1],[2],[3]] → [1,2,3]`

```
class Solution:
    def spiralOrder(self, matrix):
        if not matrix:
            return []
        l, r, t, b = 0, len(matrix[0]) - 1, 0, len(matrix) - 1
        res = []
        while True:
            for i in range(l, r + 1):
                res.append(matrix[t][i])   # 上边，从左往右
            t += 1
            if t > b:
                break
            for i in range(t, b + 1):
                res.append(matrix[i][r])   # 右边，从上往下
            r -= 1
            if l > r:
                break
            for i in range(r, l - 1, -1):
                res.append(matrix[b][i])   # 下边，从右往左
            b -= 1
            if t > b:
                break
            for i in range(b, t - 1, -1):
                res.append(matrix[i][l])   # 左边，从下往上
            l += 1
            if l > r:
                break
        return res
```

注 l、r、t、b 是未读取区域的四条闭区间边界。每走完一条边立刻收缩并检查，最后只剩一行或一列时就不会再读一遍。时间 O(m × n)，不计结果的额外空间 O(1)。

6.3 旋转图像 LC 48

给定 n × n 方阵，将它原地顺时针旋转 90°。不另建矩阵，返回 None。

例 matrix = [[1,2,3],[4,5,6],[7,8,9]] → matrix 变为 [[7,4,1],[8,5,2],[9,6,3]]，返回 None

```
class Solution:
    def rotate(self, matrix):
        n = len(matrix)
        for i in range(n):
            for j in range(i + 1, n):
                matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
        for row in matrix:
            row.reverse()                # 转置后再左右翻转每行
```

注 原位置 (i, j) 先经过转置到 (j, i)，再经过行内反转到 (j, n-1-i)。这与附件里的坐标映射相同。转置只遍历主对角线一侧，每对元素交换一次。时间 O(n²)，额外空间 O(1)。

备注 · 用新矩阵理解坐标映射

注 原稿把旧位置 (i, j) 直接写入新位置 (j, n-1-i)，旋转方向正确。matrix[:] 能更新调用者看到的列表，但已额外分配了 n × n 的矩阵。下面保留作坐标映射演示，提交时用上面的原地写法。

```
class Solution:
    def rotate(self, matrix):
        n = len(matrix)
        matrix_new = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                matrix_new[j][n - i - 1] = matrix[i][j]
        matrix[:] = matrix_new
```

6.4 搜索二维矩阵 II LC 240

给定非空矩阵，每行从左到右非递减，每列从上到下非递减。判断目标值 target 是否存在。

例 matrix = [[1,4,7],[2,5,8],[3,6,9]], target = 6 → True; target = 10 → False

```
class Solution:
    def searchMatrix(self, matrix, target):
        m, n = len(matrix), len(matrix[0])
        line, row = 0, n - 1
        while line <= m - 1 and row >= 0:
            if target < matrix[line][row]:
                row -= 1                # 当前列往下只会更大，排除这一列
            elif target > matrix[line][row]:
                line += 1               # 当前行往左只会更小，排除这一行
            else:
                return True
        return False
```

注 从右上角开始，当前值过大就左移，过小就下移。line 是行下标，row 是列下标。每步排除一行或一列，时间 O(m+n)，额外空间 O(1)。
和二分专题的 LC 74 不同，本题相邻两行的值域可以重叠，所以不能先按一行的首尾锁定唯一候选行。

专题小结

矩阵置零先做标记再修改。螺旋读取每走一边就更新边界。旋转图像用转置和行内反转。矩阵搜索从右上角逐步排除行列。

核心 先对照题意确认是否要原地改、是否能开新矩阵，以及行和列分别有哪些顺序保证。

7 链表

7.1 相交链表 LC 160

给定两个无环单链表，返回它们共享的第一个节点。不相交时返回 None。

例 A = [4,1,8,4,5]，B = [5,6,1,8,4,5]，从第一个 8 开始共用同一段节点 → 返回那个 8 的节点；A = [1]，B = [1]（两个独立节点） → None

```
class Solution:
    def getIntersectionNode(self, headA, headB):
        p, q = headA, headB
        while True:
            if p == q:
                return p              # 同一个节点，或同时为 None
            p = p.next if p else headB
            q = q.next if q else headA
```

注 两个指针走到空位就换到另一条链表。若有公共尾段，两个指针换头后会走过相同的总步数，到达同一个交点。若没有，最终同时到 None。
判题提供的 ListNode 按对象身份比较。值相等不代表相交，也不能只靠两条值序列构造相交用例。时间 O(m+n)，额外空间 O(1)。

7.2 反转链表 LC 206

给定单链表头节点 head，原地反转链表的指向，返回新的头节点。

例 head = [1,2,3] → [3,2,1]；head = [] → []

```
class Solution:
    def reverseList(self, head):
        cur, pre = head, None
        while cur:
            tmp = cur.next            # 改指针前先存好剩余链表的入口
            cur.next = pre
            pre = cur
            cur = tmp
        return pre                    # 最后处理的节点就是新头
```

注 pre 指向已反转部分的头，cur 指向下一个待处理节点。原来的头第一次反转后指向 None，最后成为尾节点。时间 O(n)，额外空间 O(1)。

7.3 回文链表 LC 234

判断单链表的节点值从前向后与从后向前是否相同。

例 head = [1,2,2,1] → True；head = [1,2,3] → False；head = [1,2,1] → True

```
import sys

if sys.getrecursionlimit() < 51000:
    sys.setrecursionlimit(51000)

class Solution:
    def isPalindrome(self, head):
        def reverse(cur, pre):
            if not cur:
                self.ans = pre        # 已走到原链尾，pre 是反转后的头
                return
            reverse(cur.next, cur)
            cur.next = pre            # 在递归返回时改指针
            return

        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        reverse(slow, None)
        a = head
        while self.ans:
            if self.ans.val == a.val:
                self.ans = self.ans.next
                a = a.next
            else:
                return False
        return True
```

注 快指针每次走两步，慢指针每次走一步。偶数长度时 slow 指向后半段的首节点，奇数长度时指向正中间，最后一次比较会用同一个中点的值。
原稿会反转后半段，比较结束后不恢复链表。时间 O(n)，反转用递归栈，额外空间 O(n)。
题面允许长链。运行这段代码时，仅在当前解释器的递归上限不足时调高它。

7.4 环形链表 LC 141

判断沿 next 一直走是否会回到已访问过的节点。

例 head = [3,2,0,-4], pos = 1 → True; head = [1], pos = -1 → False

```
class Solution:
    def hasCycle(self, head):
        cur = head
        steps = 0
        while cur:
            cur = cur.next
            steps += 1
            if steps > 10000:
                return True             # 超过题目允许的节点数，必然走过重复节点
        return False
```

注 题目保证最多 10,000 个节点。无环一定在这一步数以内走到 None，有环会继续走到第 10,001 步。pos 是判题用的环入口下标，-1 表示无环，方法只接收 head。
这份写法依赖题目的长度上限。放到任意长度的链表上，超过 10,000 个节点的无环链也会被误判。设 B 是题目给定的节点上限，最坏时间 O(B)，额外空间 O(1)。

7.5 环形链表 II LC 142

返回环开始的节点，无环时返回 None。

例 head = [3,2,0,-4], pos = 1 → 返回下标 1、值为 2 的节点; head = [1], pos = -1 → None

```
class Solution:
    def detectCycle(self, head):
        seen = set()
        cur = head
        while cur:
            if cur not in seen:
                seen.add(cur)           # 保存节点对象
            else:
                return cur              # 第一次重复访问的就是入环点
            cur = cur.next
        return None
```

注 从头走到环入口后，会先绕环一整圈，再次抵达入口。集合记录的是节点身份，相同值的不同节点可以同时存在。时间 O(n)，额外空间 O(n)。

7.6 合并两个有序链表 LC 21

把两个非递减单链表合并为一条非递减链表，返回合并后的头节点。

例 `list1 = [1,2,4], list2 = [1,3,4] → [1,1,2,3,4,4]; list1 = [], list2 = [0] → [0]`

```
class Solution:
    def mergeTwoLists(self, list1, list2):
        left, right = list1, list2
        tail = ListNode()
        dummy = tail
        while left and right:
            if left.val <= right.val:
                tail.next = left
                left = left.next
                tail = tail.next
            else:
                tail.next = right
                right = right.next
                tail = tail.next
        if left:
            tail.next = left            # 一条耗尽后，接上另一条的剩余部分
        if right:
            tail.next = right
        return dummy.next
```

注 dummy 固定留在结果头之前，tail 随着接入节点向后走。合并直接复用原节点。时间 O(m+n)，额外空间 O(1)。ListNode 由判题平台提供。

7.7 两数相加 LC 2

两个非空链表分别表示非负整数，每个节点是一位数字，最低位在链表头。按同样的逆序方式返回两数之和。

例 `l1 = [2,4,3], l2 = [5,6,4] → [7,0,8]; l1 = [9], l2 = [1] → [0,1]`

```
class Solution:
    def addTwoNumbers(self, l1, l2, s=0):
        if l1 is None and l2 is None and s == 0:
            return None
        if l1:
            s += l1.val
            l1 = l1.next
        if l2:
            s += l2.val
            l2 = l2.next
        return ListNode(s % 10, self.addTwoNumbers(l1, l2, s // 10))
```

注 进入递归时 s 是上一位的进位，加上当前两位后，s % 10 写入新节点，s // 10 传给下一层。两条链都走完仍有进位时，还要创建一个节点。
时间和递归额外空间均为 O(max(m,n))。输出使用新建节点。

7.8 删除链表的倒数第 N 个结点 LC 19

给定非空链表和合法的 n，删除倒数第 n 个节点并返回更新后的头。

例 `head = [1,2,3,4,5], n = 2 → [1,2,3,5]; head = [1], n = 1 → []`

```
class Solution:
    def removeNthFromEnd(self, head, n):
        a, b = head, head
        if head:
            x = 0
        else:
            x = -1
        while a:
            a = a.next
            x += 1

        m = x - n                       # 要删除节点的下标，从 0 开始
        if m == 0:
            return head.next            # 删头时直接返回新头
        elif m == x - 1:
            k = 1
            while k < x - 1:
                head = head.next
                k += 1
            head.next = None            # 删尾时把倒数第二个节点设为新尾
            return b
        else:
            k = 1
            while k < x - n:
                head = head.next
                k += 1
            head.next = head.next.next
            return b
```

注 先算长度 x，再把倒数第 n 个换成正向下标 x-n。删头优先判断，所以单节点链表也能正确返回 None。其他情况先走到要删除节点的前一个位置。
原稿保留了删头、删尾、删中间的三个分支。题目保证链表非空且 n 合法。时间 O(x)，额外空间 O(1)。

7.9 两两交换链表中的节点 LC 24

把链表中相邻的两个节点成对交换，返回新头。最后剩一个节点时保留原位。

例 `head = [1,2,3,4,5] → [2,1,4,3,5]; head = [] → []`

```
class Solution:
    def swapPairs(self, head):
        dummy = ListNode(0)
        dummy.next = head
        temp = dummy
        while temp.next and temp.next.next:
            node1 = temp.next
            node2 = temp.next.next
            temp.next = node2           # 前一段接到这一对的新头
            node1.next = node2.next    # 原第一节点接到下一对
            node2.next = node1
            temp = node1                # 移到这一对的新尾
        return dummy.next
```

注 每轮同时处理两个节点。虚拟头让第一对和后面的对使用相同的接线方式。改的是 next 指针。时间 O(n)，额外空间 O(1)。

7.10 K 个一组翻转链表 LC 25

每 k 个节点为一组，原地反转各组并返回新头。末尾不足 k 个节点时保持原顺序。

例 head = [1,2,3,4,5], k = 2 → [2,1,4,3,5]; k = 3 → [3,2,1,4,5]

```
import sys

if sys.getrecursionlimit() < 6000:
    sys.setrecursionlimit(6000)

class Solution:
    def reverseKGroup(self, head, k):
        x = 1
        a = head
        while x < k and a:
            a = a.next
            x += 1
        if not a:
            return head                 # 剩余节点不足 k 个

        c = a.next                      # 先保存下一组的头
        prev, cur = None, head
        for _ in range(k):
            nxt = cur.next
            cur.next = prev
            prev = cur
            cur = nxt
        head.next = self.reverseKGroup(c, k)  # 原组头成了组尾
        return prev
```

注 先检查这一组是否满 k 个，再改指针。反转后 prev 是新组头，原来的 head 变成尾，用它接上下一个递归结果。
时间 O(n)，递归深度 O(1+n/k)。k = 1 时仍然按原写法逐节点递归。
题面允许长链。运行这段代码时，仅在当前解释器的递归上限不足时调高它。

7.11 随机链表的复制 LC 138

每个节点除了 next，还带有可指向任意原节点或 None 的 random。返回深拷贝的头，复制出来的指针都指向新节点或 None。

例 head = [[7,null],[13,0]] → [[7,null],[13,0]]（新节点）；每项为 [值, random 指向的下标]

```
import sys

if sys.getrecursionlimit() < 2000:
    sys.setrecursionlimit(2000)

class Solution:
    def __init__(self):
        self.hash = {}

    def copyRandomList(self, head):
        if not head:
            return None
        if head not in self.hash:
            new_head = Node(head.val)
            self.hash[head] = new_head        # 先登记，再递归处理两条边
            new_head.next = self.copyRandomList(head.next)
            new_head.random = self.copyRandomList(head.random)
        return self.hash[head]
```

注 字典将每个原节点映射到一个新节点。random 可能回到自己或已经访问过的节点，先登记可以避免重复构造和无限递归。

独立复制时使用新的 Solution 实例。同一个实例重复传入同一原节点会命中缓存。时间和额外空间均为 O(n)。

带 random 的 Node 定义见附录。

题面允许长链。运行这段代码时，仅在当前解释器的递归上限不足时调高它。

7.12 排序链表 LC 148

将链表按节点值升序排序，返回排序后的头节点。

例 `head = [4,2,1,3] → [1,2,3,4]; head = [] → []`

```
class Solution:
    def sortList(self, head):
        nodes = []
        cur = head
        while cur:
            nodes.append(cur)          # 保存节点引用
            cur = cur.next

        new_nodes = sorted(nodes, key=lambda x: x.val)
        dummy = ListNode()
        p = dummy
        for node in new_nodes:
            p.next = node
            p = p.next
        p.next = None                  # 清掉末节点旧指针
        return dummy.next
```

注 先收集原节点，再按值排序，最后重连。必须把新的尾节点指向 None，否则可能带上旧节点，甚至形成环。

时间 O(n log n)，额外空间 O(n)。题面的常数额外空间是进阶要求，这份原解法使用了列表。

7.13 合并 K 个升序链表 LC 23

给定 k 条非递减链表，合并成一条非递减链表并返回新头。

例 `lists = [[1,4,5],[1,3,4],[2,6]] → [1,1,2,3,4,4,5,6]；lists = [] → []`

```
class Solution:
    def mergeKLists(self, lists):
        def merge(l1, l2):
            dummy = ListNode()
            cur = dummy
            while l1 and l2:
                if l1.val < l2.val:
                    cur.next = l1
                    l1 = l1.next
                else:
                    cur.next = l2
                    l2 = l2.next
                cur = cur.next
            cur.next = l1 if l1 else l2
            return dummy.next

        n = len(lists)
        mid = n // 2
        if n == 0:
            return None
        if n == 1:
            return lists[0]
        left = self.mergeKLists(lists[:mid])
        right = self.mergeKLists(lists[mid:])
        return merge(left, right)
```

注 把链表头列表分成两半，递归得到两个有序结果，再用合并两个有序链表的写法合并。递归切片复制的是头节点引用。
当 k ≥ 2 时，时间上界 O((N+k) log k)，N 是节点总数。切片峰值空间 O(k)，递归深度 O(log k)。

7.14 LRU 缓存 LC 146

实现容量固定的 LRUCache。get 取值，缺键返回 -1。put 插入或更新。每次访问都把键变为最近使用，超容量时淘汰最久未使用的键。

例 `操作 = ["LRUCache","put","put","get","put","get","get"]；参数 = [[2],[1,1],[2,2],[1],[3,3],[2],[3]] → [null,null,null,1,null,-1,3]`

```
class Node:
    def __init__(self, key=0, val=0, prev=None, next=None):
        self.key = key
        self.val = val
        self.prev = prev
        self.next = next


class LRUCache:
    def __init__(self, capacity):
```

```
        self.size = 0
        self.capacity = capacity
        self.hash = {}
        self.head = Node()
        self.tail = Node()
        self.head.next = self.tail
        self.tail.prev = self.head

    def get(self, key):
        if key not in self.hash:
            return -1
        node = self.hash[key]
        self.removenode(node)
        self.addtohead(node)                    # 访问过的节点移到头部
        return node.val

    def put(self, key, val):
        if key not in self.hash:
            node = Node(key, val)
            self.hash[key] = node
            self.addtohead(node)
            self.size += 1
            if self.size > self.capacity:
                node2 = self.removetail()     # 淘汰尾部的实际节点
                self.hash.pop(node2.key)
                self.size -= 1
        else:
            self.hash[key].val = val
            self.removenode(self.hash[key])
            self.addtohead(self.hash[key])

    def removenode(self, node):
        node.next.prev = node.prev
        node.prev.next = node.next
        node.prev = None
        node.next = None

    def addtohead(self, node):
        node.next = self.head.next
        self.head.next.prev = node
        node.prev = self.head
        self.head.next = node

    def removetail(self):
        node = self.tail.prev
        self.removenode(node)
        return node
```

注 字典按键定位节点，双向链表维护使用顺序。头尾都是哨兵，头后是最近使用，尾前是最久未用。淘汰时同时从字典和链表删掉节点。
这里的 Node 是题内自定义的双向节点，含 key、val、prev、next。get、put 平均 O(1)，空间 O(capacity)。示例的 null 对应构造或无返回值的操作。

专题小结

① 相交链表靠换头对齐路程。环形链表的计数法用到题面上限，环形链表 II 用集合找第一个重复节点。
② 反转先存 next。回文先找中点，再反转后半段比较。两两交换和 K 个一组翻转都要接好组间指针。
③ 合并有序链表每次选较小值。两数相加从低位往高位传进位。删除倒数节点先将位置换成正向下标。
④ 随机链表先建立原节点到新节点的映射。排序链表先排序节点引用再重连。合并 K 条链表先分治再两两合并。
⑤ LRU 用字典定位节点，用双向链表维护最近使用顺序。

核心 读题先分清返回的是节点还是值，是否要求原地改指针。重连前先保存后续入口。普通 ListNode、随机指针 Node 的定义见附录，LRU 的双向 Node 随题给出。

8 二叉树

8.1 二叉树的中序遍历 LC 94

给定二叉树的根节点 root，按左子树、根、右子树的顺序返回节点值。

例 `root = [1,null,2,3] → [1,3,2]；root = [] → []`

```
class Solution:
    def inorderTraversal(self, root):
        stack = []
        res = []
        while stack or root:
            if not root:
                node = stack.pop()       # 左子树已处理，可以访问该节点
                res.append(node.val)
                root = node.right
            else:
                stack.append(root)       # 先记住节点，继续向左
                root = root.left
        return res
```

注 栈里存的是还没访问的节点。走不到更左的位置时弹栈，先访问节点，再从它的右子树继续。
时间 O(n)，辅助空间 O(h)，h 为树高。

8.2 二叉树的最大深度 LC 104

返回从根到最远叶子的最长路径上的节点数。空树的深度是 0。

例 `root = [3,9,20,null,null,15,7] → 3；root = [] → 0`

```
import sys

if sys.getrecursionlimit() < 11000:
    sys.setrecursionlimit(11000)

class Solution:
    def maxDepth(self, root):
        if not root:
            return 0
        left = self.maxDepth(root.left)
        right = self.maxDepth(root.right)
        return max(left, right) + 1      # 较深的子树再加当前节点
```

注 先递归得到左右子树的深度，再取较大的值加 1。时间 O(n)，递归栈空间 O(h)。
极深的单链树可能触发 Python 的递归深度限制。代码在上限不足时调高它，保留原递归；该设置作用于当前解释器。

8.3 翻转二叉树 LC 226

交换每个节点的左右子树，原地翻转整棵树并返回根节点。

例 root = [4,2,7,1,3,6,9] → [4,7,2,9,6,3,1]；root = [] → []

```
class Solution:
    def invertTree(self, root):
        if not root:
            return None

        left = self.invertTree(root.left)
        right = self.invertTree(root.right)
        root.right = left                # 把翻转后的左子树放到右边
        root.left = right
        return root
```

注 先翻转左右子树，再交换它们的根引用。空树在 Python 中返回 None，层序示例写作 []。时间 O(n)，递归栈空间 O(h)。

8.4 对称二叉树 LC 101

判断二叉树是否沿根节点的中轴线镜像对称。

例 root = [1,2,2,3,4,4,3] → True；root = [1,2,2,null,3,null,3] → False

```
class Solution:
    def isSymmetric(self, root):
        if not root:
            return True

        def mirror(left, right):
            if not left and not right:
                return True
            if not left or not right:
                return False
            if left.val != right.val:
                return False
            # 镜像位置是左对右、右对左
            return mirror(left.left, right.right) and mirror(left.right, right.

        return mirror(root.left, root.right)
```

注 比较一对镜像节点时，先看空位是否一致，再看值是否相等。继续递归时交叉比较孩子。时间 O(n)，递归栈空间 O(h)。

8.5 二叉树的直径 LC 543

返回树上任意两个节点之间最长简单路径的长度，按边数计。路径可以不经过根。

例 root = [1,2,3,4,5] → 3；root = [1] → 0

```
import sys

if sys.getrecursionlimit() < 11000:
    sys.setrecursionlimit(11000)

class Solution:
    def diameterOfBinaryTree(self, root):
        self.ans = 0

        def dfs(node):
            if not node:
                return 0
            left = dfs(node.left)
            right = dfs(node.right)
            self.ans = max(self.ans, left + right)  # 经过当前节点的路径边数
            return max(left, right) + 1             # 返回当前子树的高度

        dfs(root)
        return self.ans
```

注 dfs 返回从当前节点向下的高度，按节点数计。经过当前节点的路径边数恰好是两侧子树高度之和。往父节点继续延伸时只能带上一侧。时间 O(n)，递归栈空间 O(h)。
极深的单链树可能触发 Python 的递归深度限制。代码在上限不足时调高它，保留原递归；该设置作用于当前解释器。

8.6 二叉树的层序遍历 LC 102

从上到下逐层访问节点，同一层从左到右。返回每层节点值组成的列表。

例 root = [3,9,20,null,null,15,7] → [[3],[9,20],[15,7]]；root = [] → []

```
class Solution:
    def levelOrder(self, root):
        if not root:
            return []

        res, queue = [], [root]
        while queue:
            tmp = []
            for node in queue:              # 这一轮遍历的是开始时的列表
                queue = queue[1:]           # 新列表去掉当前节点，再追加孩子
                tmp.append(node.val)
                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)
            res.append(tmp)
        return res
```

注 for 已经拿到本轮原列表的迭代器，queue = queue[1:] 只是把变量绑定到新列表。因此新加入的孩子留到下一轮处理。
每个节点都切片复制队列。若 w 是最大层宽，时间最坏 O(n × w)，辅助空间 O(w)。右视图题的写法可以另存下一层，省掉切片。

8.7 将有序数组转换为二叉搜索树 LC 108

给定严格升序数组 nums，构造一棵高度平衡的二叉搜索树。每个节点的左右子树高度差至多为 1。

例 nums = [-10,-3,0,5,9] → [0,-10,5,null,-3,null,9]（一种合法结果）

```
class Solution:
    def sortedArrayToBST(self, nums):
        n = len(nums)
        if n == 0:
            return None

        left, right = 0, n - 1
        mid = (left + right) // 2
        root = TreeNode(nums[mid])
        root.left = self.sortedArrayToBST(nums[:mid])
        root.right = self.sortedArrayToBST(nums[mid+1:])
        return root
```

注 每次取中点为根，左边的数更小，右边的数更大，两段长度最多差 1。TreeNode 由判题平台提供。
数组切片会复制。时间 O(n log n)，不计返回的树，切片峰值空间 O(n)，递归深度 O(log n)。

8.8 验证二叉搜索树 LC 98

判断整棵树是否满足 BST 定义。每个节点的左子树所有值都更小，右子树所有值都更大。

例 root = [2,1,3] → True；root = [2,2,3] → False

```
import sys

if sys.getrecursionlimit() < 11000:
    sys.setrecursionlimit(11000)

class Solution:
    def isValidBST(self, root):
        self.ans = []

        def dfs(node):
            if not node:
                return
            dfs(node.left)
            self.ans.append(node.val)
            dfs(node.right)

        dfs(root)
        for i in range(1, len(self.ans)):
            if self.ans[i] <= self.ans[i-1]:
                return False              # 中序序列必须严格递增
        return True
```

注 看整棵树的中序序列。相邻值相等也不合法；只检查当前节点和它的两个直接孩子会漏掉更深处的违规值。
时间 O(n)，辅助空间 O(n)。
极深的单链树可能触发 Python 的递归深度限制。代码在上限不足时调高它，保留原递归；该设置作用于当前解释器。

8.9 二叉搜索树中第 K 小的元素 LC 230

给定 BST 和合法的 k，返回按值从小到大排在第 k 位的节点值。k 从 1 开始。

例 `root = [3,1,4,null,2], k = 1 → 1; k = 3 → 3`

```
import sys

if sys.getrecursionlimit() < 11000:
    sys.setrecursionlimit(11000)

class Solution:
    def kthSmallest(self, root, k):
        self.k = k
        self.ans = 0

        def dfs(node):
            if not node:
                return
            dfs(node.left)
            if self.k > 1:
                self.k -= 1
            elif self.k == 1:
                self.ans = node.val
                self.k -= 1               # 变为 0，通知上层已经找到
                return
            else:
                return
            dfs(node.right)               # 只有还没找到才继续搜右子树

        dfs(root)
        return self.ans
```

注 中序遍历依次访问从小到大的值。k 每访问一个节点就减 1，减到 0 时保存答案。外层先运行 dfs，再返回 self.ans。时间 O(h + k)，递归栈空间 O(h)。

极深的单链树可能触发 Python 的递归深度限制。代码在上限不足时调高它，保留原递归；该设置作用于当前解释器。

8.10 二叉树的右视图 LC 199

从树的右侧观察，返回每一层最右边节点的值，按从上到下的顺序排列。

例 root = [1,2,3,null,5,null,4] → [1,3,4]；root = [] → []

```
class Solution:
    def rightSideView(self, root):
        if not root:
            return []
        queue = [root]
        ans = []
        k = 0
        while queue:
            tmp = []
            for i in range(len(queue)):
                node = queue[i]
                if i == len(queue) - 1:
                    ans.append(node.val)    # 当前层最后一个节点
                if node.left:
                    tmp.append(node.left)
                if node.right:
                    tmp.append(node.right)
            queue = tmp
            k += 1
        return ans
```

注 每层先把左孩子放入下一层，再放右孩子。因此每层最后访问的是最右节点。k 只记录层数，没有参与求解。时间 O(n)，辅助空间 O(w)，w 为最大层宽。

8.11 二叉树展开为链表 LC 114

按前序顺序把原树展开成一条只走 right 指针的链。所有 left 都为 None，函数返回 None。

例 root = [1,2,5,3,4,null,6] → 原树变为 [1,null,2,null,3,null,4,null,5,null,6]，返回 None

```
import sys

if sys.getrecursionlimit() < 3000:
    sys.setrecursionlimit(3000)

class Solution:
    def flatten(self, root):
        self.ans = []

        def dfs(node):
            if not node:
                return
            self.ans.append(node)            # 保存原节点，保留前序顺序
            dfs(node.left)
            dfs(node.right)

        dfs(root)
        for i in range(len(self.ans) - 1):
            self.ans[i].left = None
            self.ans[i].right = self.ans[i+1]  # 指向前序中的下一个节点
```

注 先记住原节点，再逐一改指针。前序的最后一个节点必然是叶子，原来的左右指针都已经是 None。
方法原地修改节点，省略 return 即返回 None。时间 O(n)，辅助列表和递归栈合计 O(n)。
极深的单链树可能触发 Python 的递归深度限制。代码在上限不足时调高它，保留原递归；该设置作用于当前解释器。

8.12 从前序与中序遍历序列构造二叉树 LC 105

给定同一棵树的前序序列 preorder 和中序序列 inorder，节点值互不相同。重建原树并返回根节点。

例 `preorder = [3,9,20,15,7], inorder = [9,3,15,20,7] → [3,9,20,null,null,15,7]`

```
import sys

if sys.getrecursionlimit() < 4000:
    sys.setrecursionlimit(4000)

class Solution:
    def buildTree(self, preorder, inorder):
        val_to_idx = {v: i for i, v in enumerate(inorder)}

        def dfs(pre_l, pre_r, in_l, in_r):
            if pre_l > pre_r:
                return None
            root_val = preorder[pre_l]        # 前序首元素是根
            root_in = val_to_idx[root_val]
            left_cnt = root_in - in_l         # 中序中根左边都是左子树
            root = TreeNode(root_val)
            root.left = dfs(pre_l+1, pre_l+left_cnt, in_l, root_in-1)
            root.right = dfs(pre_l+left_cnt+1, pre_r, root_in+1, in_r)
            return root

        return dfs(0, len(preorder)-1, 0, len(inorder)-1)
```

注 四个下标表示前序和中序的两个闭区间。先在中序中算出左子树节点数，再用这个数量把前序分成根、左子树、右子树三段。
字典让根在中序中的下标可以直接查到。时间 O(n)，不计结果树的额外空间 O(n)。
极深的单链树可能触发 Python 的递归深度限制。代码在上限不足时调高它，保留原递归；该设置作用于当前解释器。

8.13 路径总和 III LC 437

统计节点值总和等于 targetSum 的向下路径。路径只沿父节点走向子节点，起终点可以是任意节点。

例 `root = [10,5,-3,3,2,null,11,3,-2,null,1]`, `targetSum = 8` → `3`; `root = []`, `targetSum = 0` → `0`

```
import sys

if sys.getrecursionlimit() < 2000:
    sys.setrecursionlimit(2000)

from collections import defaultdict

class Solution:
    def pathSum(self, root, targetSum):
        self.hash = defaultdict(int)
        self.ans = 0
        self.cur = 0
        self.hash[0] = 1                # 根节点之前有一个空前缀

        def dfs(node):
            if not node:
                return
            self.cur += node.val
            self.ans += self.hash[self.cur - targetSum]
            self.hash[self.cur] += 1     # 查完再登记，避免把空路径计进去
            dfs(node.left)
            dfs(node.right)
            self.hash[self.cur] -= 1     # 离开时撤销前缀记录
            self.cur -= node.val

        dfs(root)
        return self.ans
```

注 若当前前缀和是 cur，祖先路径上每出现一次 cur - targetSum，就对应一条以当前节点结尾的合法路径。进入节点时登记，退出时减回去，左右分支的前缀不会串起来。计数为 0 的键仍留在字典中，辅助空间最坏 O(n)，时间 O(n)。

极深的单链树可能触发 Python 的递归深度限制。代码在上限不足时调高它，保留原递归；该设置作用于当前解释器。

8.14 二叉树的最近公共祖先 LC 236

给定二叉树和其中两个不同的节点 p、q，返回它们最深的公共祖先。节点可以是自己的祖先。

例 root = [3,5,1,6,2,0,8,null,null,7,4]，p = 5，q = 4 → 值为 5 的原节点

```
import sys

if sys.getrecursionlimit() < 101000:
    sys.setrecursionlimit(101000)

class Solution:
    def lowestCommonAncestor(self, root, p, q):
        self.path1 = []
        self.path2 = []
        self.cur = []

        def dfs(node):
            if not node:
                return
            self.cur.append(node)
            if node == p:
                self.path1 = self.cur[:]   # 保存从根到 p 的路径
            if node == q:
                self.path2 = self.cur[:]
            dfs(node.left)
            dfs(node.right)
            self.cur.pop()

        dfs(root)
        ans = None
        for i in range(min(len(self.path1), len(self.path2))):
            if self.path1[i] == self.path2[i]:
                ans = self.path1[i]       # 保留当前最深的共同节点
            else:
                break
        return ans
```

注 从根到 p、q 的两条路径都从同一个根出发。比较共同前缀，最后一个相同节点就是答案。题目保证 p、q 存在，示例用值指明节点，函数实际返回节点对象。时间 O(n)，辅助空间 O(h)。
极深的单链树可能触发 Python 的递归深度限制。代码在上限不足时调高它，保留原递归；该设置作用于当前解释器。

8.15 二叉树中的最大路径和 LC 124

返回二叉树中任意一条非空简单路径的最大节点值之和。路径可以不经过根。

例 root = [-10,9,20,null,null,15,7] → 42；root = [-3] → -3

```
import sys

if sys.getrecursionlimit() < 31000:
    sys.setrecursionlimit(31000)

class Solution:
    def maxPathSum(self, root):
        max_sum = -10**9

        def dfs(node):
            nonlocal max_sum
            if not node:
                return 0
            left = max(dfs(node.left), 0)
            right = max(dfs(node.right), 0)
            max_sum = max(max_sum, node.val + left + right)  # 此处可接两边
            return node.val + max(left, right)               # 向上只能接一边

        dfs(root)
        return max_sum
```

注 和直径题一样，要分开看「在当前节点合并两侧」和「向父节点上报一侧」。负收益取 0 表示不接那条子路径。

路径至少有一个节点，答案不能从 0 起步。题目保证节点值不低于 -1000，原稿的 -10**9 足够小。时间 O(n)，递归栈空间 O(h)。

极深的单链树可能触发 Python 的递归深度限制。代码在上限不足时调高它，保留原递归；该设置作用于当前解释器。

专题小结

① 中序遍历用栈保存还未访问的节点。最大深度向上返回较大高度加 1。翻转先处理子树再交换，对称则交叉比较两边。

② 直径在当前节点合并两侧高度。最大路径和合并两侧非负收益，向父节点只能交出一侧。

③ 层序遍历收集每层所有节点，右视图只收每层最后一个。

④ 有序数组建 BST 取中点。验证 BST 检查中序是否严格递增，第 K 小在中序中按访问次序计数。

⑤ 展开用前序保存原节点，再改指针。前序加中序建树时，先从中序算左子树大小。

⑥ 路径总和 III 用当前前缀减目标值查次数。最近公共祖先找两条根路径的最后一个共同节点。

核心 先区分要返回节点、节点值还是 None。写递归时先确定返回给父节点的量，再判断哪些答案需要在当前节点更新。树的示例统一用层序数组表示，null 是缺失的子节点，末尾的 null 可以省略。

9 图论 4/4

9.1 岛屿数量 LC 200

给定由字符 "1"（陆地）和 "0"（水）组成的网格。上下左右相邻的陆地属于同一座岛，返回岛屿数量。

例 `grid = [["1","0"],["0","1"]] → 2（对角不相连）；grid = [["0"]] → 0`

```
class Solution:
    def numIslands(self, grid):
        m = len(grid)
        n = len(grid[0])

        def dfs(grid, x, y):
            stack = [(x, y)]
            grid[x][y] = 0                    # 入栈时就标记，避免重复入栈
            while stack:
                x, y = stack.pop()
                for a, b in [(x-1, y), (x+1, y), (x, y-1), (x, y+1)]:
                    if 0 <= a < m and 0 <= b < n and grid[a][b] == "1":
                        grid[a][b] = 0
                        stack.append((a, b))

        ans = 0
        for i in range(m):
            for j in range(n):
                if grid[i][j] == "1":
                    dfs(grid, i, j)         # 一次 DFS 标记整座岛
                    ans += 1
        return ans
```

注 扫描时第一次遇到某座岛，就用 DFS 标记所有相连的陆地。代码沿用原稿，用整数 0 原地标记已访问的陆地，后续只认字符串 "1"。
原稿的递归 DFS 在岛很长时可能触发 Python 的递归深度限制。显式栈仍做深度优先搜索，可处理题目允许的大网格。
时间 O(m × n)，辅助栈最坏 O(m × n)。

9.2 腐烂的橘子 LC 994

网格中 0 是空格，1 是新鲜橘子，2 是腐烂橘子。每分钟，所有腐烂橘子同时感染上下左右相邻的新鲜橘子。返回全部腐烂所需的分钟数，无法全部腐烂时返回 -1。

例 `grid = [[2,1,1],[1,1,0],[0,1,1]] → 4; grid = [[1,0,2]] → -1; grid = [[0]] → 0`

```
class Solution:
    def orangesRotting(self, grid):
        m = len(grid)
        n = len(grid[0])
        num = 0
        queue = []
        ans = 0
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 1:
                    num += 1
                elif grid[i][j] == 2:
                    queue.append([i, j])    # 所有烂橘子同时作为起点

        while queue:
            tmp = []
            for a, b in queue:
                for x, y in [(a-1, b), (a+1, b), (a, b-1), (a, b+1)]:
                    if 0 <= x < m and 0 <= y < n and grid[x][y] == 1:
                        grid[x][y] = 2      # 立即标记，避免同一格重复计数
                        tmp.append([x, y])
                        num -= 1
            queue = tmp
            if tmp:
                ans += 1                    # 本轮确实有新橘子变烂
        return ans if not num else -1
```

注 这是多源 BFS。queue 放本轮要扩散的橘子，tmp 放这一轮刚变烂的橘子。tmp 要等到下一轮才继续扩散。
num 始终是剩余新鲜橘子的数量。BFS 结束后还有新鲜橘子就返回 -1。代码会原地更新 grid。时间 O(m × n)，辅助空间 O(m × n)。

备注 · BFS 层数和分钟数

注 错点是把分钟数从 -1 开始，每处理一层就加 1。只要初始队列有烂橘子，这种写法可以抵消最后一轮没有新感染的加 1。可是全空网格 [[0]] 没有初始烂橘子，也没有新鲜橘子，循环一次都不会进，分钟数一直是 -1，正确结果是 0。改进是从 0 开始，只在 tmp 非空时加 1，让分钟数直接表示发生了几轮感染。以后先用 [[0]]、[[2]]、[[1]]、[[2,1]] 手推，答案分别是 0、0、-1、1。检查初始队列为空、最后一轮没感染、只扩散一轮这几种情况。

9.3 课程表 LC 207

共有 numCourses 门课，编号从 0 到 numCourses - 1。先修关系 [a,b] 表示先上 b 才能上 a。判断能否修完全部课程。

例 numCourses = 2, prerequisites = [[1,0]] → True; prerequisites = [[1,0],[0,1]] → False; prerequisites = [] → True

```
from collections import defaultdict

class Solution:
    def canFinish(self, numCourses, prerequisites):
        indegree = defaultdict(int)
        next_courses = defaultdict(list)
        ans = 0
        queue = []
        for course, prerequisite in prerequisites:
            indegree[course] += 1
            next_courses[prerequisite].append(course)  # 边：先修课 → 后续课
        for course in range(numCourses):
            if indegree[course] == 0:
                ans += 1
                queue.append(course)

        while queue:
            tmp = []
            for course in queue:
                for nxt in next_courses[course]:
                    indegree[nxt] -= 1
                    if indegree[nxt] == 0:
                        ans += 1            # 所有先修课都已完成
                        tmp.append(nxt)
            queue = tmp
        return ans == numCourses
```

注 入度表示还有多少先修课没有完成。先把入度为 0 的课放入队列。处理完一门课，就把它指向的课程的入度减 1。

每门课在第一次变为可修时计数一次。队列空了却还有课程没计入，说明剩余依赖中有环。时间和辅助空间都是 O(V + E)，V 是课程数，E 是先修关系数。

9.4 实现 Trie (前缀树) LC 208

实现 Trie，支持插入单词、查询完整单词、查询是否存在给定前缀。

例 操作 = ["Trie","insert","search","search","startsWith","insert","search"]；参数 = [[],["apple"],["apple"],["app"],["app"],["app"],["app"]] → [null,null,true,false,true,null,true]

```
class Trie:
    def __init__(self):
        self.root = Node()

    def insert(self, word):
        cur = self.root
        for ch in word:
            if ch not in cur.son:
                cur.son[ch] = Node()        # 缺少这条字符边就新建节点
            cur = cur.son[ch]
        cur.end = True                      # 整个单词到这里结束

    def search(self, word):
        cur = self.root
        for ch in word:
            if ch not in cur.son:
                return False
            else:
                cur = cur.son[ch]
        return True if cur.end else False

    def startsWith(self, prefix):
        cur = self.root
        for ch in prefix:
            if ch not in cur.son:
                return False
            else:
                cur = cur.son[ch]
        return True                         # 前缀路径走通即可


class Node:
    def __init__(self):
        self.son = dict()
        self.end = False
```

注 son 把下一个字符映射到对应子节点，end 表示某个已插入的单词在这一格结束。只插入 apple 时，app 的路径存在，但 app 的 end 仍是 False。
insert 会在原 Trie 上添加节点或标记词尾，返回 None。示例里的 null 对应构造或无返回值的操作。三种操作的时间都是 O(L)，L 是本次传入的单词或前缀长度。

专题小结

① 岛屿数量　遇到一块未访问陆地，就用 DFS 标记整座岛。
② 腐烂的橘子　所有初始烂橘子一起做 BFS，下一层有新感染才加一分钟。
③ 课程表　从入度为 0 的课程出发，逐步解除后续课程的先修限制。
④ 前缀树　顺着字符走，查完整单词还要检查词尾标记。

核心　网格题先定邻接方向和标记时机。分层搜索要分清队列里放的是哪一轮，以及哪次状态变化才应计入答案。

10 回溯

10.1 全排列 LC 46

给定互不相同的整数数组 nums，返回由全部元素组成的所有排列。答案顺序不限。

例 nums = [1,2,3] → [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

```
class Solution:
    def permute(self, nums):
        path, ans = [], []

        def dfs():
            if len(path) == len(nums):
                ans.append(path[:])     # 保存快照，后续还要修改 path
                return
            for num in nums:
                if num not in path:    # 当前排列里还没用过
                    path.append(num)
                    dfs()
                    path.pop()          # 撤销这次选择

        dfs()
        return ans
```

注 每层决定排列的下一位。每一层都从整个 nums 里选，只跳过当前路径已经用过的值。这里用值判断是否用过，依赖 nums 中的元素互不相同。
路径长到 n 才收集答案。列表里的成员检查需要扫描当前路径，时间上界 $O(n^2 \times n!)$，不计答案的额外空间 $O(n)$。

10.2 子集 LC 78

给定互不相同的整数数组 nums，返回它的所有子集，包括空集。每个子集只返回一次，答案顺序不限。

例 nums = [1,2] → [[],[1],[1,2],[2]]

```
class Solution:
    def subsets(self, nums):
        n = len(nums)
        path, ans = [], []

        def dfs(start):
            ans.append(path[:])          # 到达每个节点时，路径已经是一个子集
            for i in range(start, n):
                path.append(nums[i])
                dfs(i + 1)               # 下一次只能从更靠后的下标选
                path.pop()

        dfs(0)
        return ans
```

注 根节点的路径为空，所以空集也会被收集。每层枚举下一个要选的下标，递归传 i+1。同一批元素只会按下标从小到大选出一次。
全排列只在选满时收集，子集每到一个节点就收集。复制所有子集的总时间 $O(n \times 2^n)$，不计答案的额外空间 $O(n)$。

10.3 电话号码的字母组合 LC 17

给定由数字 2 到 9 组成的非空字符串 digits，每位数字按电话键盘对应字母。每位选一个字母，返回所有组合。

例 `digits = "23" → ["ad","ae","af","bd","be","bf","cd","ce","cf"]；digits = "2" → ["a","b","c"]`

```
class Solution:
    def letterCombinations(self, digits):
        letters = {
            "2": "abc", "3": "def", "4": "ghi", "5": "jkl",
            "6": "mno", "7": "pqrs", "8": "tuv", "9": "wxyz",
        }
        n = len(digits)
        path, ans = [], []

        def dfs(i):
            if i == n:
                ans.append("".join(path))  # 各位选完后再拼成字符串
                return
            for ch in letters[digits[i]]:
                path.append(ch)
                dfs(i + 1)                 # 处理下一位数字
                path.pop()

        dfs(0)
        return ans
```

注 递归下标 i 指向当前处理的数字。每层只在这一位数字对应的字母里选。
每位最多有 4 种选择，时间上界 $O(n \times 4^n)$，不计答案的额外空间 $O(n)$。

10.4 组合总和 LC 39

给定互不相同的正整数 candidates 和正整数 target，返回所有和恰好为 target 的组合。同一个候选数可以重复选，组合顺序不限。

例 `candidates = [2,3,6,7], target = 7 → [[2,2,3],[7]]；candidates = [2], target = 1 → []`

```python
class Solution:
    def combinationSum(self, candidates, target):
        n = len(candidates)
        path, ans = [], []

        def dfs(start):
            total = sum(path)
            if total == target:
                ans.append(path[:])
                return
            if total > target:            # 候选数全是正数，再选只会更大
                return
            for i in range(start, n):
                path.append(candidates[i])
                dfs(i)                    # 允许下一次继续选 candidates[i]
                path.pop()

        dfs(0)
        return ans
```

注 下一次从 i 开始，允许重复使用当前数。下标不会往回走，所以 [2,2,3] 和 [3,2,2] 只会收集其中一种。主解不排序。每进一层计算一次当前总和，超出 target 才返回。下面的剪枝版提前排除选了就会超出的分支。

备注 · 排序后按剩余目标剪枝

注 两种方法都枚举下一个数，并允许重复使用。主解用 sum(path) 计算已选总和，尝试选中后才知道是否超标。这里把剩余目标 left 传下去，省去重复求和。候选数排序后，发现当前数大于 left，就能跳过它和后面所有数。比如剩余 2，遇到 3 就直接结束本层循环。sort() 会原地改变 candidates 的顺序。

```python
class Solution:
    def combinationSum(self, candidates, target):
        candidates.sort()
        path, ans = [], []

        def dfs(start, left):
            if left == 0:
                ans.append(path.copy())
                return
            for i in range(start, len(candidates)):
                num = candidates[i]
                if num > left:
                    break                 # 排过序，后面的数也太大
                path.append(num)
                dfs(i, left - num)        # 仍可选当前数，剩余目标减去 num
                path.pop()

        dfs(0, target)
        return ans
```

10.5 括号生成 LC 22

给定正整数 n，返回用 n 对圆括号组成的所有合法字符串。答案顺序不限。

例 n = 3 → ["((()))","(()())","(())()","()(())","()()()"]；n = 1 → ["()"]

```
class Solution:
    def generateParenthesis(self, n):
        path, ans = [], []

        def dfs(i, left, right):
            if i == 2 * n:
                ans.append("".join(path))
                return
            if left < n:                    # 左括号还有剩余
                path.append("(")
                dfs(i + 1, left + 1, right)
                path.pop()
            if left > right:                # 有未配对的左括号才能放右括号
                path.append(")")
                dfs(i + 1, left, right + 1)
                path.pop()

        dfs(0, 0, 0)
        return ans
```

注 left、right 分别是已经放进去的左右括号数。任意前缀都要满足 right ≤ left ≤ n。两种条件都满足时要分别试，所以用两个 if。

路径达到 2n 时左右括号必然各有 n 个。不计答案的额外空间 O(n)。

10.6 单词搜索 LC 79

给定非空的字符网格 board 和非空单词 word，判断是否能从某格出发，沿上下左右相邻格依次拼出 word。同一条路径中每个格子最多使用一次。

例 board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]]，word = "ABCCED" → True；word = "ABCB" → False

```python
class Solution:
    def exist(self, board, word):
        m, n = len(board), len(board[0])
        visited = [[False] * n for _ in range(m)]
        self.ans = False

        def dfs(row, col, i):
            if board[row][col] != word[i]:
                return
            if i == len(word) - 1:
                self.ans = True
                return
            visited[row][col] = True    # 当前路径已经使用这一格
            for r, c in ((row-1, col), (row+1, col), (row, col-1), (row, col+1)):
                if 0 <= r < m and 0 <= c < n and not visited[r][c]:
                    dfs(r, c, i + 1)
                    if self.ans:
                        return          # 已经找到，结束整次搜索
            visited[row][col] = False   # 这一条路失败，撤销访问标记

        for row in range(m):
            for col in range(n):
                if board[row][col] == word[0]:
                    dfs(row, col, 0)
                    if self.ans:
                        return True
        return False
```

注 递归下标 i 指向这一格要匹配的字符。主解用独立的 visited 矩阵防止同一路径重复使用格子，用 self.ans 把找到答案的消息传回上层。
这一条路失败时解除访问标记，其他起点和路径还可以用这格。找到答案后会结束整次搜索，局部 visited 可以直接丢弃，棋盘本身没有被修改。

备注 · 字符计数、反向起搜与原地标记

注 主解直接枚举起点，用独立矩阵标记访问状态，并用 self.ans 报告成功。优化版先数棋盘里每种字符够不够用。够用时再比较单词两端，若末尾字符在棋盘上更少，就把单词反过来，从更少的候选起点搜。路径能反向走，所以答案不变。递归直接返回布尔值，临时把已访问格改成空字符串。失败时恢复该格，找到答案就立即返回。因此成功时棋盘上的部分格子会保留为空字符串，之后复用棋盘时要留意。Counter 的逐项比较需要 Python 3.10 或更新版本。

```
from collections import Counter

class Solution:
    def exist(self, board, word):
        counts = Counter(ch for row in board for ch in row)
        if not (counts >= Counter(word)):
            return False                    # 某种字符不够用
        if counts[word[-1]] < counts[word[0]]:
            word = word[::-1]               # 从棋盘上更少见的那一端起搜

        m, n = len(board), len(board[0])

        def dfs(row, col, i):
            if board[row][col] != word[i]:
                return False
            if i == len(word) - 1:
                return True
            board[row][col] = ""            # 临时标记这一格已用过
            for r, c in ((row, col-1), (row, col+1), (row-1, col), (row+1, col)
                if 0 <= r < m and 0 <= c < n and dfs(r, c, i + 1):
                    return True
            board[row][col] = word[i]       # 失败后恢复这一格
            return False

        return any(dfs(r, c, 0) for r in range(m) for c in range(n))
```

10.7 分割回文串 LC 131

给定非空字符串 s，把它切成若干连续子串，每个子串都必须是回文。返回所有合法切法，答案顺序不限。

例 s = "aab" → [["a","a","b"],["aa","b"]]；s = "a" → [["a"]]

```
class Solution:
    def partition(self, s):
        n = len(s)
        path, ans = [], []

        def dfs(i, pending):
            piece = pending + s[i]       # 把当前字符接到尚未切下的一段
            is_palindrome = piece == piece[::-1]
            if i == n - 1:
                if is_palindrome:        # 到末尾必须切下最后一段
                    path.append(piece)
                    ans.append(path[:])
                    path.pop()
                return

            if is_palindrome:
                path.append(piece)       # 选择在这里切
                dfs(i + 1, "")
                path.pop()
            dfs(i + 1, piece)            # 选择先不切，下一位继续拼

        dfs(0, "")
        return ans
```

注 path 保存已经切好的回文段，pending 保存还没切的那一段。每个字符后都有机会切或不切。只有当前段为回文时才能切。
当前段即使不是回文，也要保留继续拼的分支。例如 "ab" 还不是回文，再接一个 "a" 就成了 "aba"。最后一个字符后没有继续拼的选项。

10.8 N 皇后 LC 51

在 n × n 棋盘放置 n 个皇后，任意两个皇后不能同行、同列或同对角线。返回所有布局，每个布局用 n 个字符串表示，Q 是皇后，. 是空格。

例 n = 4 → [[".Q..","...Q","Q...","..Q."],["..Q.","Q...","...Q",".Q.."]]；n = 1 → [["Q"]]

```
class Solution:
    def solveNQueens(self, n):
        cols, diag_sum, diag_diff = set(), set(), set()
        board = [["."] * n for _ in range(n)]
        ans = []

        def dfs(row, col):
            if row == n:
                ans.append(["".join(line) for line in board])
                return
            if col in cols or row + col in diag_sum or row - col in diag_diff:
                return
            board[row][col] = "Q"
            cols.add(col)
            diag_sum.add(row + col)
            diag_diff.add(row - col)

            for next_col in range(n):
                if row == n - 1:
                    dfs(row + 1, next_col)  # 棋盘已放满，只记录一次
                    break
                dfs(row + 1, next_col)      # 下一行逐列试放

            board[row][col] = "."
            cols.remove(col)
            diag_sum.remove(row + col)
            diag_diff.remove(row - col)

        for col in range(n):
            dfs(0, col)
        return ans
```

注 每次递归试放当前行的一个位置，下一层换到下一行，所以不会同行。同列的 col 相同，两种对角线分别用 row+col、row-col 判断。
最后一行放好后，只调用一次下一层。下一层发现 row == n，就保存当前棋盘。回溯时移走皇后，并从三个集合里撤销对应记录。

专题小结

① 全排列　每层选下一位，选满才收集。
② 子集　每到一个节点就收集，从 i+1 继续选。
③ 电话号码　每层选一位数字的一个字母。
④ 组合总和　从 i 继续选，允许重复使用。
⑤ 括号生成　左括号有余额就能放，右括号必须有左括号可配。
⑥ 单词搜索　沿相邻格匹配，只限制当前路径的访问。
⑦ 分割回文串　每位决定切不切，只有回文才能切。
⑧ N 皇后　逐行放，用列和两种对角线排除冲突。

核心　写递归前先定好每层做什么选择、什么时候收集答案。退出某个分支后如果还要继续搜，要恢复共用的路径、标记和棋盘。保存列表答案时要复制。

11 二分查找 6/6

11.1 二分查找（闭区间模板） LC 704 非 Hot100

升序无重复数组 nums 中找 target，返回下标，不存在返回 -1。

例 `nums = [-1,0,3,5,9,12], target = 9 → 4 (nums[4] 就是 9)`

```
class Solution:
    def search(self, nums, target):
        left, right = 0, len(nums) - 1   # 搜索区间 [left, right]，两端都可能是答案
        while left <= right:             # 闭区间非空就继续
            mid = left + (right - left) // 2
            if nums[mid] == target:
                return mid
            if nums[mid] < target:
                left = mid + 1           # mid 已判过，排除
            else:
                right = mid - 1
        return -1
```

注 闭区间模板。`[left, right]` 中的两个端点都还可能是答案，循环条件是 `left <= right`。
命中就返回 mid。中点值偏小，丢掉 mid 及它左边的部分，写 `left = mid + 1`。中点值偏大，写 `right = mid - 1`。区间为空就返回 -1。

11.2 搜索插入位置 LC 35

给定升序且无重复元素的数组 nums 和目标值 target。找到就返回下标，否则返回按升序插入的位置。

例 `nums = [1,3,5,6], target = 5 → 2; target = 2 → 1; target = 7 → 4`

```
class Solution:
    def searchInsert(self, nums, target):
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = (left + right) // 2
            if target < nums[mid]:
                right = mid - 1
            elif target > nums[mid]:
                left = mid + 1
            else:
                return mid
        return left  # 没找到时，left 就是插入位置
```

注 和模板一样，搜索闭区间 `[left, right]`，命中立即返回下标。模板找不到时返回 -1，本题返回 `left`。
找不到时，循环结束于 `left == right + 1`。left 左边的值都比 target 小，从 left 开始的值都比 target 大。插在最前面返回 0，插在末尾返回 len(nums)。
时间 O(log n)，额外空间 O(1)。

11.3 搜索二维矩阵 LC 74

给定非空矩形矩阵 matrix，每行非递减，下一行的首元素严格大于上一行的尾元素。判断 target 是否在矩阵中。

例 matrix = [[1,3,5],[7,9,11]], target = 9 → True；target = 4 → False；target = 6 → False

```
class Solution:
    def searchMatrix(self, matrix, target):
        m, n = len(matrix), len(matrix[0])
        top, bottom = 0, m - 1
        while top <= bottom:
            mid = (top + bottom) // 2
            if target < matrix[mid][0]:
                bottom = mid - 1     # 目标在本行首元素之前
            elif target > matrix[mid][n-1]:
                top = mid + 1        # 目标在本行尾元素之后
            else:
                top = mid            # 唯一可能含有目标的行
                break
        if top > bottom:
            return False

        left, right = 0, n - 1
        while left <= right:
            mid = (left + right) // 2
            if target == matrix[top][mid]:
                return True
            if target < matrix[top][mid]:
                right = mid - 1
            else:
                left = mid + 1
        return False
```

注 模板直接在一个数组里找下标。本题先用行首、行尾对行号做闭区间二分，再按模板在选中的行里找元素，最后返回布尔值。

落在某行首尾之间，只说明目标可能在这一行。比如 4 落在 [1,3,5] 的首尾之间，但行里没有 4。若目标落在两行之间的空档，第一轮就找不到候选行。

时间 O(log m + log n)，额外空间 O(1)。

11.4 在排序数组中查找元素的第一个和最后一个位置 LC 34

给定非递减数组 nums 和目标值 target，返回 target 首次和最后一次出现的下标。目标不存在或数组为空时，返回 [-1,-1]。

例 `nums = [5,7,7,8,8,10], target = 8 → [3,4]; target = 6 → [-1,-1]; nums = [], target = 0 → [-1,-1]`

```
class Solution:
    def searchRange(self, nums, target):
        def first_position():
            found = False
            left, right = 0, len(nums) - 1
            while left <= right:
                mid = (left + right) // 2
                if nums[mid] == target:
                    found = True
                    right = mid - 1     # 命中后继续向左找
                elif nums[mid] > target:
                    right = mid - 1
                else:
                    left = mid + 1
            return left if found else -1

        def last_position():
            found = False
            left, right = 0, len(nums) - 1
            while left <= right:
                mid = (left + right) // 2
                if nums[mid] == target:
                    found = True
                    left = mid + 1      # 命中后继续向右找
                elif nums[mid] > target:
                    right = mid - 1
                else:
                    left = mid + 1
            return right if found else -1

        return [first_position(), last_position()]
```

注 模板命中就返回。本题允许重复，命中时先记下 found，再继续缩区间。找第一个位置时把相等归到左搜，退出后 left 指向第一个大于等于 target 的位置。找最后一个位置时把相等归到右搜，退出后 right 指向最后一个小于等于 target 的位置。
两个边界下标只有在找到过 target 时才是答案。found 为 False 就返回 -1，空数组也走这一分支。
两次二分合计 O(log n)，额外空间 O(1)。

11.5 搜索旋转排序数组 LC 33

给定一个由无重复的升序数组旋转得到的数组 nums，也可能未旋转。返回 target 的下标，不存在时返回 -1。

例 `nums = [4,5,6,7,0,1,2], target = 0 → 4; target = 3 → -1`

```
class Solution:
    def search(self, nums, target):
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = (left + right) // 2
            if nums[mid] == target:
                return mid
            if nums[left] <= nums[mid]:          # 左半段有序
                if nums[left] <= target < nums[mid]:
                    right = mid - 1
                else:
                    left = mid + 1
            else:                                # 右半段有序
                if nums[mid] < target <= nums[right]:
                    left = mid + 1
                else:
                    right = mid - 1
        return -1
```

注 和模板一样用闭区间，命中返回下标，找不到返回 -1。模板的整个区间都升序，比较 nums[mid] 和 target 就能选方向。旋转后要先用端点找出有序半段，再看 target 是否落在这半段的值域内。
有序半段的外侧端点要计入值域。mid 已经判过，所以靠 mid 的一侧用严格不等号。无重复元素保证这里能判断哪一半有序。
时间 O(log n)，额外空间 O(1)。

11.6 寻找旋转排序数组中的最小值 LC 153

给定非空数组 nums，它由无重复的升序数组旋转得到，也可能未旋转。返回最小元素。

例 `nums = [4,5,6,7,0,1,2] → 0；nums = [1,2,3] → 1；nums = [7] → 7`

```
class Solution:
    def findMin(self, nums):
        left, right = 0, len(nums) - 1
        while left < right:                    # 只剩一个候选位置时结束
            mid = (left + right) // 2
            if nums[mid] <= nums[right]:
                right = mid                    # mid 本身可能是最小值
            else:
                left = mid + 1                 # 右边至少有一个更小的值
        return nums[left]
```

注 模板比较 nums[mid] 和 target。本题没有 target，改比 nums[mid] 和 nums[right]。中点值比右端小，说明中点到右端这一段有序，最小值在中点或其左边。中点值比右端大，说明最小值在中点右边。

模板两边都排除已检查的 mid。本题缩右端时保留 mid，写 `right = mid`。循环用 `left < right`，候选缩到一个位置就返回该位置的值。

时间 O(log n)，额外空间 O(1)。

11.7 寻找两个正序数组的中位数 LC 4

给定两个非递减数组 nums1、nums2，至少有一个非空。返回合在一起后的中位数。总长度为奇数时取中间值，为偶数时取中间两值的平均数。

例 nums1 = [1,3], nums2 = [2] → 2；nums1 = [1,2], nums2 = [3,4] → 2.5；nums1 = [], nums2 = [4] → 4

```
class Solution:
    def findMedianSortedArrays(self, nums1, nums2):
        m, n = len(nums1), len(nums2)
        half = (m + n + 1) // 2              # 左半边需要放的元素个数

        def val(nums, i):
            if i < 0:
                return float("-inf")         # 左边没有元素
            if i >= len(nums):
                return float("inf")          # 右边没有元素
            return nums[i]

        left, right = 0, m - 1
        while left <= right:
            mid = (left + right) // 2
            take1 = mid + 1
            take2 = half - take1
            if val(nums1, mid) <= val(nums2, take2):
                left = mid + 1               # 还能尝试从 nums1 多取一些
            else:
                right = mid - 1              # nums1 取太多，向左收缩

        take1 = right + 1                    # nums1 最多能放进左半边的个数
        take2 = half - take1
        left_max = max(val(nums1, take1 - 1), val(nums2, take2 - 1))
        if (m + n) % 2 != 0:
            return left_max
        right_min = min(val(nums1, take1), val(nums2, take2))
        return (left_max + right_min) / 2
```

注 模板二分的是 target 的下标。本题二分的是分割位置。若从 nums1 左边取 i 个，就必须从 nums2 左边取 j = half - i 个。两个左段合起来放满左半边。

比较 nums1 左边最后一个和 nums2 右边第一个。前者 ≤ 后者时，继续尝试多取一个。随着 i 增大，前者不会变小、后者不会变大，所以能用二分找到最后一个满足条件的位置。如果 nums1 还能取，再取一个就会让 nums1 右边第一个大于 nums2 左边最后一个，因此当前分割也满足另一侧的大小关系。若 nums1 已取完，它的右边按正无穷处理。

和模板命中即返回不同，本题循环结束后用 right + 1 得到 nums1 左段长度。缺少左侧元素按负无穷处理，缺少右侧元素按正无穷处理。奇数取左半最大值，偶数再和右半最小值求平均。

在 nums1 上做二分，耗时 O(1 + log(m + 1))，额外空间 O(1)。nums1 为空时直接由边界算出结果。

专题小结

① 搜索插入位置　没找到时返回 left。
② 搜索二维矩阵　先找可能的行，再在行内查找。
③ 查找首尾位置　命中后继续向对应的边界收缩。
④ 搜索旋转排序数组　先识别有序半段，再按值域选方向。
⑤ 找旋转数组的最小值　和右端比较，向左收缩时保留 mid。
⑥ 寻找两个正序数组的中位数　二分左半边从第一个数组取多少个。

核心　以 LC 704 的闭区间为起点。写每一道变形前，先确定二分找的是什么，mid 是否仍可能是答案，以及循环结束后该返回什么。

12 栈

12.1 有效的括号 LC 20

给定只含 ()、[]、{} 六种字符的字符串 s，判断括号是否全部按正确类型和嵌套顺序闭合。

例 `s = "([])" → True; s = "([)]" → False; s = "(" → False`

```
class Solution:
    def isValid(self, s):
        pairs = {"(": ")", "[": "]", "{": "}"}
        stack = []
        for ch in s:
            if ch in pairs:
                stack.append(ch)  # 左括号入栈，等待闭合
            elif not stack or pairs[stack.pop()] != ch:
                return False      # 无左括号可配对，或栈顶类型不匹配
        return not stack          # 所有左括号都必须已闭合
```

注 右括号只能匹配最近一个尚未闭合的左括号。`not stack or ...` 先判断栈空，短路时不会调用 `pop()`。

时间 O(n)，额外空间 O(n)。

12.2 最小栈 LC 155

实现 MinStack，支持入栈、出栈、查看栈顶和取最小值，每个操作要求 O(1)。pop、top、getMin 只在栈非空时调用。

例 操作 = ["MinStack","push","push","push","getMin","pop","getMin","pop","top"];
参数 = [[],[3],[1],[1],[],[],[],[],[]] → [null,null,null,null,1,null,1,null,3]
（null 对应构造或无返回值的操作）

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, value):
        if not self.min_stack or self.min_stack[-1] >= value:
            self.min_stack.append(value)  # 相等的最小值也分别入栈
        self.stack.append(value)

    def pop(self):
        value = self.stack.pop()
        if value == self.min_stack[-1]:
            self.min_stack.pop()          # 同步移除这一次最小值

    def top(self):
        return self.stack[-1]

    def getMin(self):
        return self.min_stack[-1]
```

注 辅助栈的栈顶始终是当前最小值。两个 1 都入栈时，弹出一个 1 后另一个 1 仍是最小值，所以条件要包含相等。
各操作摊还 O(1)，其中 top、getMin 直接取末尾，为 O(1)。辅助栈额外空间最坏 O(n)。

12.3 字符串解码 LC 394

给定合法编码字符串 s，k[片段] 表示将片段重复 k 次。k 为正整数，可以有多位，方括号可以嵌套。返回解码后的字符串。

例 `s = "3[a2[c]]" → "accaccacc"; s = "10[a]b" → "aaaaaaaaaab"`

```
class Solution:
    def decodeString(self, s):
        stack = []
        cur = ""
        num = 0
        for ch in s:
            if ch.isdigit():
                num = num * 10 + int(ch)  # 累积多位重复次数
            elif ch == "[":
                stack.append((cur, num))   # 保存外层已解码前缀和重复次数
                cur = ""
                num = 0
            elif ch == "]":
                prev, k = stack.pop()
                cur = prev + cur * k      # 展开当前层，接回外层前缀
            else:
                cur += ch
        return cur
```

注 `cur` 是当前层已解码的内容，`num` 是下一个左方括号前的重复次数。读到右方括号时，先完成最近打开的一层。

题目中的数字是 0 到 9，可直接用 `isdigit()` 和 `int(ch)`。字符串重复和拼接会复制字符，嵌套时同一段内容可能复制多次，耗时要计入这些复制。

12.4 每日温度 LC 739

给定每天的温度 temperatures，返回每一天要等几天才会遇到严格更高的温度。之后没有更高温度时填 0。

例 `temperatures = [73,74,75,71,69,72,76,73] → [1,1,4,2,1,1,0,0]；temperatures = [70,70,71] → [2,1,0]`

```
class Solution:
    def dailyTemperatures(self, temperatures):
        stack = []
        ans = [0] * len(temperatures)
        for i, temp in enumerate(temperatures):
            while stack and temp > stack[-1][0]:
                _, prev_i = stack.pop()    # 今天是 prev_i 右侧第一个更暖的日子
                ans[prev_i] = i - prev_i
            stack.append([temp, i])        # 记录还在等待升温的日期
        return ans
```

注 这里在等更暖的一天，栈内温度从底到顶越来越低，允许相等。新温度高于栈顶就弹出，直到栈空或栈顶温度大于等于新温度，再把今天入栈。

比如栈里从底到顶是 `[76, 73, 71]`，今天是 74。先弹 71，再弹 73，因为 74 比它们都高。遇到 76 停下，入栈后是 `[76, 74]`。今天就是被弹出那两天右边第一次升温的日子。

相同温度不算升温。遍历完还在栈里的日期没有等到更高温度，答案保留为 0。每个下标至多入栈和出栈各一次，时间 O(n)，辅助空间 O(n)。

12.5 柱状图中最大的矩形 LC 84

给定非负整数数组 heights，每根柱子宽度为 1。返回由连续柱子围成的最大矩形面积，矩形高度不能超过覆盖范围内的最低柱子。

例 `heights = [2,1,5,6,2,3]` → 10（高度 5、宽度 2）；`heights = [2,2,2]` → 6

```
class Solution:
    def largestRectangleArea(self, heights):
        n = len(heights)
        left = [-1] * n
        right = [n] * n
        stack = []
        for i, h in enumerate(heights):
            while stack and h < stack[-1][1]:
                j, _ = stack.pop()
                right[j] = i                # j 右侧第一个严格更矮的位置
            if stack:
                left[i] = stack[-1][0]      # i 左侧最近一个不高于 h 的位置
            stack.append([i, h])            # 栈内高度从底到顶非递减
        ans = 0
        for i in range(n):
            width = right[i] - left[i] - 1  # 两侧边界都不计入宽度
            ans = max(ans, heights[i] * width)
        return ans
```

注 递增单调栈从栈底往栈顶看，数值从小到大。本题允许相等，准确说是非递减。新柱子比栈顶矮，就先把栈顶弹掉。一直弹到栈空，或剩下的栈顶不高于新柱子，才让新柱子入栈。弹出的都比新柱子高，留下的都不高于新柱子。

比如栈里从底到顶是 `[2, 4, 6, 8]`，新柱子高度是 5。先弹 8，再弹 6，因为它们都比 5 高。遇到 4 停下，因为 `4 <= 5`。把 5 放进去，栈就变成 `[2, 4, 5]`。

对弹出的 8 和 6，右边第一个严格更矮的柱子都是 5。对新入栈的 5，左边最近一个不高于它的柱子是留下来的栈顶 4。找不到左、右边界时，分别用 -1 和 n。

本题只弹出比新柱子高的柱子，等高会一起留在栈里。后面的等高柱可能被左边的同高柱挡住。看任意一个矩形，取其中最靠左的最低柱子，它算出的宽度就能覆盖整个矩形。

每根柱子至多入栈和出栈各一次。时间 O(n)，辅助空间 O(n)。

专题小结

① 有效的括号　左括号入栈，右括号匹配栈顶。

② 最小栈　辅助栈保存当前及历史最小值，相等的最小值也入栈。

③ 字符串解码　遇到左方括号保存外层状态，遇到右方括号展开并接回。

④ 每日温度　当前温度严格更高时弹栈，等待天数是两个下标之差。

⑤ 最大矩形　当前柱子严格更矮时弹栈，宽度是 `right[i] - left[i] - 1`。

核心 先确定栈里保存什么，再写出何时弹栈。单调栈题要分清比较是否包含相等，以及遍历结束时栈内元素的含义。

13 堆

13.1 数组中的第 k 个最大元素 LC 215

返回 nums 排序后第 k 大的元素（按位置算，不是第 k 个不同值）。

例 nums = [3,2,1,5,6,4], k = 2 → 5（降序 6,5,4,3,2,1，第 2 个是 5）

```
import heapq

class Solution:
    def findKthLargest(self, nums, k):
        h = []                                  # 小根堆，只留当前最大的 k 个
        for i in nums:
            if len(h) < k:
                heapq.heappush(h, i)            # 前 k 个先填满
            else:
                if i > h[0]:                    # 堆顶是这 k 个里最小的，比它大才换
                    heapq.heappop(h)
                    heapq.heappush(h, i)
        return h[0]                             # 堆顶就是第 k 大
```

13.2 前 K 个高频元素 LC 347

返回 nums 中出现频率最高的 k 个元素，顺序任意。

例 nums = [1,1,1,2,2,3], k = 2 → [1,2]（1 出现 3 次，2 出现 2 次）

```
import heapq
from collections import defaultdict

class Solution:
    def topKFrequent(self, nums, k):
        freq = defaultdict(int)
        for num in nums:
            freq[num] += 1                      # 缺键自动补 0，可以直接 +1

        h = []                                  # 小根堆存 (频率, 元素)，堆顶是 k 个里最
        for num, count in freq.items():
            if len(h) < k:
                heapq.heappush(h, (count, num))
            else:
                if h[0][0] < count:             # 比堆顶频率高才够格替换
                    heapq.heapreplace(h, (count, num))
        return [num for count, num in h]        # 题目允许任意顺序，不用再排
```

13.3 数据流的中位数 LC 295

数据流里不断加入整数，随时返回当前所有元素的中位数。

例 addNum(1) → addNum(2) → findMedian() = 1.5；再 addNum(3) → findMedian() = 2.0

```
import heapq

class MedianFinder:
    def __init__(self):
        self.l = []                 # 大根堆（存负数），装较小的一半
        self.r = []                 # 小根堆，装较大的一半

    def addNum(self, num):
        heapq.heappush(self.l, -num)              # 先无脑进左堆
        if len(self.l) - len(self.r) > 1:         # 左边多出 2 个，挪一个过去
            a = heapq.heappop(self.l)
            heapq.heappush(self.r, -a)
        if self.l and self.r and -self.l[0] > self.r[0]:    # 左堆最大 > 右堆最小
            b = heapq.heappop(self.l)             # 交换两个堆顶，恢复有序
            c = heapq.heappop(self.r)
            heapq.heappush(self.r, -b)
            heapq.heappush(self.l, -c)

    def findMedian(self):
        m = len(self.l)
        n = len(self.r)
        if (m + n) % 2 == 0:                      # 偶数个，两个堆顶取平均
            return (self.r[0] - self.l[0]) / 2    # l 存的是负数，所以是相减
        else:                                     # 奇数个，左堆多一个
            return -self.l[0]
```

专题小结

① 第 k 大：完全不用把数排好，只要盯住「目前最大的那几个」，让这几个里最差的那个守在门口，新来的比它强就换掉它，最后站在门口的就是第 k 大。
② 前 k 高频：先数一遍每个数出现几次，再用同一招只留「出现最多的 k 个」，只不过比较的是次数、不是数值本身。
③ 中位数：把所有数从中间掰成两半，小的一半放左边、大的一半放右边，只要两边数量差不超过一个，中位数就永远卡在两半的交界上。

核心 堆只维护局部顺序，便于反复取最小或最大边界。第 k 大与前 k 高频用大小至多为 k 的小根堆淘汰候选，中位数用两个堆维护两半的边界和数量。小 k 或持续插入时很合适；不保证任何规模下都比一次排序快。

14 贪心算法

14.1 买卖股票的最佳时机 LC 121

给定每天股价 prices，只能买卖一次（先买后卖），返回能拿到的最大利润；赚不到就返回 0。

例 prices = [7,1,5,3,6,4] → 5（第 2 天 1 买入，第 5 天 6 卖出）

```
class Solution:
    def maxProfit(self, prices):
        n = len(prices)
        a = prices[-1]                    # 循环开始时 a = i 之后的最高价
        ans = 0
        for i in range(n - 2, -1, -1):    # 从倒数第二天倒着往前扫
            b = a - prices[i]             # 第 i 天买、后面最高价卖，能赚多少
            ans = max(ans, b)             # ans 初值 0，亏损自然被挡掉
            a = max(a, prices[i])         # 把第 i 天也算进后缀最高价
        return ans
```

14.2 跳跃游戏 LC 55

给定每个位置能跳的最大步数 nums，从下标 0 出发，判断能不能到达最后一个下标。

例 nums = [2,3,1,1,4] → true；nums = [3,2,1,0,4] → false（下标 3 只能跳 0 步，卡死）

```
class Solution:
    def canJump(self, nums):
        maxpos = 0                              # 当前能到达的最远下标
        for i in range(len(nums)):
            if i > maxpos:                      # 连 i 都到不了，后面更到不了
                return False
            maxpos = max(maxpos, i + nums[i]) # 用 i 起跳，更新最远可达
            if maxpos >= len(nums) - 1:         # 已经够到最后一位，直接赢
                return True
```

14.3 跳跃游戏 II LC 45

同上一个数组 nums，返回跳到最后一个下标所需的最少跳跃次数；题目保证一定到得了。

例 nums = [2,3,1,1,4] → 2（0 → 1 → 4，两步到位）

```
class Solution:
    def jump(self, nums):
        ans = 0                                  # 已跳的步数
        end = 0                                  # 当前这一跳能覆盖的右边界
        maxpos = 0                               # 下一跳能到达的最远处
        for i in range(len(nums) - 1):           # 不用看最后一位，站在它上面就结束了
            maxpos = max(maxpos, i + nums[i]) # 边扫边扩下一跳的范围
            if i == end:                         # 走到本层边界，必须再跳一次
                ans += 1
                end = maxpos
        return ans
```

14.4 划分字母区间 LC 763

把字符串切成尽量多的段，同一个字母只能出现在一段里，返回每段的长度。

例 s = "ababcbacadefegdehijhklij" → [9,7,8]（切成 ababcbaca / defegde / hijhklij）

```
class Solution:
    def partitionLabels(self, s):
        start = -1                       # 上一段的结尾，第一段之前当作 -1
        maxpos = 0                       # 当前这一段至少要延伸到哪
        ans = []
        last = {c: i for i, c in enumerate(s)}   # 每个字母最后一次出现的下标
        for i in range(len(s)):
            if i > maxpos:               # 越过上一段结尾，上一段可以封闭了
                ans.append(maxpos - start)   # 段长 = 本段结尾 - 上段结尾
                start = maxpos
            if i == len(s) - 1:          # 最后一段没有下一段来触发，就地收尾
                ans.append(i - start)
            j = last[s[i]]
            maxpos = max(maxpos, j)      # 本段必须盖住 s[i] 最后的出现位置
        return ans
```

注 j 的两种找法，效果完全一样：

哈希版 `j = last[s[i]]` —— 先花 O(n) 建一次表，之后每个字符平均 O(1) 查，总 O(n)；代价是多一个 dict。

库函数版 `j = s.rfind(s[i])` —— 不建表，每次在整串中倒着查找，单次 O(n)，整体最坏 O(n²)；代码少一行，n 小的时候随便用。

专题小结

① 买卖股票：与其对每一天都往后找最高价，不如倒着走，一边走一边记住「后面见过的最高价」，拿它减去当天的价钱，就是这天买入能赚多少。
② 跳跃游戏：不用管具体跳哪几步，只跟住「目前最远能到哪儿」，路过一个位置就试着把这个距离往外推一推；一旦有个位置自己都够不着，后面全废。
③ 最少步数：在上一题基础上多记一件事——这一跳管到哪儿为止，走到边界就必须再跳，跳一次就往外扩一层，扩了几层就是走了几步。
④ 划分区间：既然一个字母不许跨两段，这一段就得盖住段内所有字母的最后出现位置，所以边扫边把结尾往后推，推不动了就切一刀。

核心 这四题都能用已扫描的信息更新状态，但状态不只一个数。股票维护后缀最高价与答案；跳跃游戏维护最远可达位置；最少跳数还维护当前层边界和步数；划分区间先记录每个字母的最后位置，再维护本段边界。贪心需要证明局部选择不会损害全局结果，不能只看题目是否问最值。例如硬币 1、3、4 凑 6，优先拿最大面额需 3 枚，最优却是两个 3。

15 动态规划 10/10

15.1 爬楼梯 LC 70

共有 n 级台阶，每次可以爬 1 级或 2 级，返回爬到楼顶的不同走法数（n 为正整数）。

例 n = 3 → 3（1+1+1、1+2、2+1 是三种不同走法）

```
class Solution:
    def climbStairs(self, n):
        dp = [0] * (n + 1)              # dp[i] 表示爬到第 i 级台阶的走法数
        dp[0] = 1                       # 不走任何一步算一种空方案，作为递推起点
        dp[1] = 1                       # 爬到第 1 级只有一种走法
        for i in range(2, n + 1):        # 从小到大填表，自下而上计算
            dp[i] = dp[i - 1] + dp[i - 2]  # 转移 dp[i] = dp[i-1] + dp[i-2]
        return dp[n]                    # 爬到第 n 级的走法总数
```

注 自下而上：先确定 `dp[0]`、`dp[1]`，再依次算到 `dp[n]`。时间 O(n)，额外空间 O(n)。

15.2 杨辉三角 LC 118

给定行数 numRows，返回杨辉三角的前 numRows 行；每行两端都是 1，中间每个数等于上一行相邻两个数之和。

例 numRows = 3 → [[1],[1,1],[1,2,1]]

```
class Solution:
    def generate(self, numRows):
        ans = []                            # 保存每一行，行号从 0 开始
        for i in range(numRows):
            ans.append([1] * (i + 1))        # 第 i 行有 i+1 个数，先全部填成 1
        for i in range(2, numRows):          # 前两行已正确，从第三行开始补中间
            for j in range(1, i):           # 跳过每行两端，端点始终为 1
                # 转移 ans[i][j] = ans[i-1][j-1] + ans[i-1][j]
                ans[i][j] = ans[i-1][j-1] + ans[i-1][j]
        return ans                          # 返回包含所有行的二维列表
```

注 每行都用 `[1] * (i + 1)` 单独创建，避免多行共用同一个列表。时间和保存结果所需的空间均为 O(numRows²)。

15.3 打家劫舍 LC 198

非空数组 nums 记录一排房屋中的非负金额，不能同时偷相邻两间房，返回能偷到的最大总金额。

例 nums = [2,7,9,3,1] → 12（偷第 1、3、5 间，2+9+1 = 12）

```
class Solution:
    def rob(self, nums):
        dp = [0] * (len(nums) + 1)      # dp[k] 是前 k 间房的最大金额，dp[0] 为 0
        dp[1] = nums[0]                 # 前 1 间房的答案，对应 nums 的下标 0
        for i in range(1, len(nums)):   # i 是房屋下标，从第 2 间开始处理
            # 转移 dp[i+1] = max(dp[i], dp[i-1] + nums[i])
            dp[i+1] = max(dp[i], dp[i-1] + nums[i])
        return dp[-1]                   # 前 len(nums) 间，也就是全部房屋的答案
```

注 dp 比 nums 多一位：nums[i] 是第 i+1 间房，dp[i] 只考虑前 i 间，dp[i+1] 才包含当前房。不偷当前房取 dp[i]；偷当前房取 dp[i-1] + nums[i]，避免偷相邻房屋。
边界是 dp[0] = 0、dp[1] = nums[0]；题目保证 nums 非空，只有一间时循环不执行，直接返回 dp[1]。时间 O(n)，额外空间 O(n)。

15.4 完全平方数 LC 279

给定正整数 n，返回和为 n 的完全平方数的最少个数；同一个平方数可以重复使用。

例 n = 12 → 3 (4+4+4) ; n = 13 → 2 (4+9)

```
import math

class Solution:
    def numSquares(self, n):
        dp = [float("inf")] * (n + 1)  # dp[i] 是凑出 i 的最少个数，先设为无穷大
        dp[0] = 0                      # 凑出 0 不需要任何平方数
        for i in range(1, n + 1):       # 从小到大填表，自下而上计算
            for j in range(1, math.isqrt(i) + 1):  # 枚举不超过 i 的平方数 j**2
                # 转移 dp[i] = 1 + min(dp[i-j**2])，遍历 1 <= j <= isqrt(i)
                dp[i] = min(dp[i], dp[i - j**2] + 1)
        return dp[-1]                  # 最后一项就是凑出 n 的最少个数
```

注 `math.isqrt(i)` 是向下取整的整数平方根；range 右端加 1 才能枚举到它。`j ** 2` 是 j 的平方，`^` 是异或，不能替代乘方。

下面的 DP 类封装写法仍按 j = 1..n 计算，属于自下而上，不是记忆化递归。`get()` 是这里自定义的方法；负下标返回无穷大，用来排除超过当前金额的平方数。两种写法的时间均为 O(n√n)，额外空间均为 O(n)。

补充写法：DP 类封装（仍为自下而上）

```
import math

class Solution:
    def numSquares(self, n):
        class DP:
            def __init__(self, n):
                self.dp = [0] * (n + 1)  # 保存各个状态，dp[0] 为 0
            def get(self, x):
                if x < 0:
                    return float("inf")  # 余数为负，这个平方数不能选
                else:
                    return self.dp[x]    # 返回此前已经算出的状态
        dp = DP(n)
        m = math.isqrt(n)                # 枚举的平方根上界
        for j in range(1, n + 1):        # 仍然从小状态算到大状态
            # 转移（取所有合法剩余金额的最小值，再加 1）
            dp.dp[j] = min(dp.get(j - i**2) for i in range(1, m + 1)) + 1
        return dp.dp[n]                  # 每次转移相当于再选一个平方数
```

15.5 零钱兑换 LC 322

给定正整数硬币面额数组 coins 和非负金额 amount，每种硬币可无限使用；返回凑出 amount 所需的最少硬币数，无法凑出则返回 -1。

例 coins = [1,2,5], amount = 11 → 3 (5+5+1) ; coins = [2], amount = 3 → −1; amount = 0 → 0

```
class Solution:
    def coinChange(self, coins, amount):
        dp = [float("inf")] * (amount + 1)  # dp[i] 是凑出金额 i 的最少硬币数
        dp[0] = 0                           # 凑出 0 元不需要硬币
        for i in range(1, amount + 1):       # 从小金额开始，自下而上填表
            for j in coins:                 # j 是硬币面额，不是数组下标
                if j <= i:                  # 面额不超过当前金额，才考虑选它
                    # 转移 dp[i] = 1 + min(dp[i-j]), 遍历 coins 中 j <= i
                    dp[i] = min(dp[i-j] + 1, dp[i])
        return dp[amount] if dp[amount] != float("inf") else -1  # 无解时返回 -1
```

注 dp[i-j] 中可能已经使用过面额 j，因此同一种硬币可以重复选。无穷大表示不可达状态，加 1 后仍为无穷大，不会覆盖有限的最优解。
amount = 0 时循环不执行，直接返回 dp[0] = 0。时间 O(amount × len(coins))，额外空间 O(amount)。

15.6 单词拆分 LC 139

给定字符串 s 和词典 wordDict，判断能否按原顺序用词典中的单词拼成整个 s；单词可以重复使用，不能遗漏字符。

例 s = "leetcode", wordDict = ["leet","code"] → True (leet+code) ; 同一词典下 s = "leetcodes" → False

```
class Solution:
    def wordBreak(self, s, wordDict):
        n = len(s)
        dp = [False] * (n + 1)          # dp[i] 表示前 i 个字符 s[:i] 能否拼出
        dp[0] = True                    # 空前缀可以拼出，作为递推起点
        for i in range(1, n + 1):       # 从短前缀开始，自下而上判断
            for word in wordDict:       # 尝试把 word 放在当前前缀的末尾
                j = i - len(word)       # 若末尾放 word，它的起点就是 j
                # 转移 dp[i] = OR(dp[j] and s[j:i] == word), 仅取 j >= 0
                if j >= 0 and dp[j] and s[j:i] == word:
                    dp[i] = True        # 前 j 个字符能拼出，末尾单词也完全匹配
                    break               # 当前前缀已可拼出，不用再试其他单词
        return dp[n]                    # 判断整个字符串，而不只是其中一段
```

注 dp 长度为 n+1，索引表示字符个数。s[j:i] 左闭右开，长度为 i-j；先检查 j >= 0，再判断前缀 dp[j] 和末尾单词是否匹配。
设 n 为字符串长度、m 为词典大小、L 为最长单词长度，最坏时间 O(n × m × L)，其中包含切片和字符串比较的耗时；空间 O(n)。

15.7 最长递增子序列 LC 300

给定整数数组 nums，返回最长严格递增子序列的长度；可以跳过元素，但必须保留原有顺序，相等的数不能接在一起。

例 `nums = [10,9,2,5,3,7,101,18] → 4（例如 2,3,7,101）；nums = [2,2,2] → 1`

```
class Solution:
    def lengthOfLIS(self, nums):
        n = len(nums)
        dp = [1] * n                    # dp[i] 是以 nums[i] 结尾的最长长度，至少有自己
        for i in range(n):              # 依次确定每个结尾位置的答案
            for j in range(i):          # 尝试接在前面的 nums[j] 后面
                if nums[i] > nums[j]:   # 必须严格变大，相等不能接
                    # 转移 dp[i] = max({1} ∪ {dp[j]+1 | j < i 且 nums[j] < nums[i
                    dp[i] = max(dp[i], dp[j] + 1)
        return max(dp) if nums else 0   # 最优序列可在任意位置结尾；空数组返回 0
```

注 `dp[i]` 必须以 `nums[i]` 结尾，不是整个前缀的最优长度。例如 nums = [1,2,3,0] 时，dp = [1,2,3,1]，答案是 `max(dp) = 3`，不是最后一项。

时间 O(n²)，额外空间 O(n)。空数组时 dp 也为空，先判断 nums，避免对空列表调用 max。

15.8 乘积最大子数组 LC 152

给定非空整数数组 nums，找出乘积最大的非空连续子数组，返回这个最大乘积；数组可含负数和 0。

例 nums = [2,3,-2,4] → 6（子数组 [2,3]）；nums = [-2,3,-4] → 24（整段相乘）

```
class Solution:
    def maxProduct(self, nums):
        n = len(nums)
        min_n = [0] * n                 # min_n[i] 是以 nums[i] 结尾的最小乘积
        max_n = [0] * n                 # max_n[i] 是以 nums[i] 结尾的最大乘积
        min_n[0] = nums[0]              # 非空子数组，第一项只能取 nums[0]
        max_n[0] = nums[0]
        for i in range(1, n):
            # 令 x = nums[i], C = {x, x*max_n[i-1], x*min_n[i-1]}
            # 转移 max_n[i] = max(C), min_n[i] = min(C)
            max_n[i] = max(nums[i], nums[i] * max_n[i-1], nums[i] * min_n[i-1])
            min_n[i] = min(nums[i], nums[i] * max_n[i-1], nums[i] * min_n[i-1])
        return max(max_n)               # 最优子数组可在任意位置结尾
```

注 负数会让最大、最小乘积互换角色，所以两个状态都要保存。转移时还要考虑只取当前元素，重新开始一段。数组版时间 O(n)，额外空间 O(n)。
优化版先用旧状态算出 a、b，再一起更新 max_n、min_n，不能让 b 用到已更新的 max_n。按下标遍历，避免 nums[1:] 复制列表；时间 O(n)，额外空间 O(1)。

空间优化：只保留上一位置的最大、最小乘积

```
class Solution:
    def maxProduct(self, nums):
        max_n = min_n = ans = nums[0]  # 当前结尾的最大、最小乘积，以及全局答案
        for i in range(1, len(nums)):  # 用下标遍历，不创建 nums[1:] 的临时列表
            x = nums[i]
            a = max(x, x * max_n, x * min_n)  # 转移（用旧状态计算新的最大乘积）
            b = min(x, x * max_n, x * min_n)  # 转移（用旧状态计算新的最小乘积）
            max_n = a                 # 两个新值都算完后，再更新状态
            min_n = b
            ans = max(ans, max_n)       # 更新全局答案，不只看最后一个位置
        return ans
```

15.9 分割等和子集 LC 416

给定非空正整数数组 nums，判断能否分成两个元素和相等的子集；每个位置的数只能分到一边。等价于寻找总和一半的子集。

例 nums = [1,5,11,5] → True（[11] 与 [1,5,5]）；nums = [1,2,5] → False

```python
class Solution:
    def canPartition(self, nums):
        total = sum(nums)
        if total % 2:                          # 总和为奇数，不可能平分
            return False
        target = total // 2                    # 只需凑出总和的一半
        dp = [False] * (target + 1)             # dp[j] 表示已处理的数能否凑出和 j
        dp[0] = True                           # 不选任何数，可以凑出 0
        for num in nums:                       # 每个位置对应一件只能选一次的物品
            for j in range(target, num - 1, -1):  # 金额倒序，防止重复使用当前物品
                # 转移 D(k,j) = D(k-1,j) or D(k-1,j-num)，此处 j >= num
                dp[j] = dp[j] or dp[j - num]
        return dp[target]                      # 能凑出一半，剩下的数就构成另一半
```

注 0/1 背包：每个数组位置是一件物品，重量是该数，目标容量为 target；每件只能选 0 次或 1 次。相同数值出现在不同位置，仍算不同物品。这里求「能否恰好装满」，所以用布尔状态和 or 转移。
为什么倒序：在外层枚举物品、一维压缩时，倒序保证 `dp[j-num]` 仍是加入当前物品之前的状态。同一轮中不会借用刚更新的较小金额，因此不会重复选 num。num > target 时，这个 range 为空，自动跳过。
与完全背包的区别：完全背包每件物品可重复选。在同样「外层物品、一维状态」的写法下，内层金额正序，使 `dp[j-num]` 可包含本轮物品。若把本题改成正序，[1,2,5] 会误用多个 1 凑出 4，错误返回 True。
与零钱兑换的联系：都是凑目标金额；零钱兑换允许面额重复、求最少个数，用 min，本题每个位置只能用一次、求可行性，用 or。循环顺序须结合状态定义；零钱兑换原解按金额再枚举面额，同样能求最少个数。
设 n 为数组长度，target 为总和的一半：时间 O(n × target)，额外空间 O(target)。下面两份 DFS 仅作思路备注，未记忆化，最坏是指数级搜索。

备注一：DFS 枚举子集，使用 nonlocal 传出结果

注 `nonlocal Flag` 声明要重新绑定最近一层外部函数已有的 Flag；不写时，内层的 `Flag = True` 只会建立局部变量，外层仍为 False。它不是模块级 global。 `path.append()` / `path.pop()` 修改同一个列表，不重新绑定 path，因此不需要 nonlocal；若要在内层执行 `path = []`，才需声明 nonlocal 才能替换外层绑定。 此版 DFS 不做搜索剪枝，每个叶子还要 sum(path)，最坏时间 $O(n \times 2^n)$，辅助空间 O(n)，大输入可能超时。

```python
class Solution:
    def canPartition(self, nums):
        a = sum(nums)
        n = len(nums)
        if a % 2 != 0:                  # 总和为奇数，不可能平分
            return False
        a = a // 2                      # 子集的目标和，使用整数整除
        path = []                       # 当前选入子集的数
        Flag = False                    # 是否找到目标子集
        def dfs(x):
            nonlocal Flag               # 修改外层 canPartition 中的 Flag
            if x == n:                  # 每个位置都已决定选或不选
                if sum(path) == a:
                    Flag = True         # 找到一个和为目标值的子集
                return
            dfs(x + 1)                  # 不选 nums[x]
            path.append(nums[x])        # 选择 nums[x]
            dfs(x + 1)
            path.pop()                  # 撤销选择，还原当前路径
        dfs(0)
        return Flag
```

备注二：DFS 累计和、剪枝和短路返回

> 注 用参数 s 记录当前和，返回 bool，不需要 nonlocal。s > 目标和的剪枝依赖正整数条件；or 左支为 True 时不再搜索右支。未记忆化，最坏时间 $O(2^n)$，辅助空间 O(n)，大输入可能超时。

```python
class Solution:
    def canPartition(self, nums):
        a = sum(nums)
        n = len(nums)
        if a % 2 != 0:                  # 奇数总和不能平分
            return False
        a = a // 2
        def dfs(x, s):                  # x 是待处理下标，s 是已选数字之和
            if s == a:
                return True             # 已凑出目标，无需再选剩余数字
            if x == n or s > a:         # 数已用完，或正整数累加后超过目标
                return False
            return dfs(x + 1, s) or dfs(x + 1, s + nums[x])
        return dfs(0, 0)                # 先尝试不选，再尝试选，or 会短路
```

15.10 最长有效括号 LC 32

给定只含左括号和右括号的字符串 s，返回最长连续有效括号子串的长度；要求括号正确配对，不能跳过中间字符。

例 s = ")()())" → 4（子串 ()()）；s = "(()(()" → 2；s = "" → 0

```
class Solution:
    def longestValidParentheses(self, s):
        n = len(s)
        dp = [0] * (n + 1)        # dp[i] 是前 i 个字符的最长有效括号后缀长度
        for i in range(1, n + 1):
            if s[i - 1] == ")":   # 只有右括号可以结束有效括号段
                j = i - dp[i - 1] - 2  # 跳过已有后缀，找当前右括号对应的左括号
                if j >= 0 and s[j] == "(":
                    dp[i] = dp[j] + 2 + dp[i - 1]  # 转移 dp[i] = dp[j] + 2 + dp
        return max(dp)             # 最长有效段可以在任意位置结束
```

注 设当前右括号在下标 i-1。先跳过前一位置的有效后缀，再找候选左括号下标 `j = i - dp[i-1] - 2`。配对成功时，`dp[i] = dp[j] + 2 + dp[i-1]`。三项分别是左括号前相邻的有效段、新配成的一对、被包住的有效段。其余位置保持 0，答案取 `max(dp)`。
时间 O(n)，额外空间 O(n)。空串时 dp 只有初始的 0。

备注 · 栈与匹配标记

注 栈存未匹配的左括号下标。配对后标记两端，再取最长连续 True 段。未匹配位置会切断子串。时间 O(n)，额外空间 O(n)。

```
class Solution:
    def longestValidParentheses(self, s):
        stack = []                      # 保存尚未匹配的左括号下标
        ans = [False] * len(s)          # True 表示该位置已与另一个括号配对
        path = 0                        # 当前连续匹配段的长度
        res = 0                         # 已找到的最长长度
        for i, x in enumerate(s):
            if x == "(":
                stack.append(i)         # 左括号先入栈，等待右括号
            elif x == ")" and stack:
                a = stack.pop()         # 与最近一个未匹配的左括号配对
                ans[a] = True           # 标记这一对括号的两个位置
                ans[i] = True
        for j in range(len(ans)):       # 再找标记中最长的连续 True 段
            if ans[j]:
                path += 1
                res = max(res, path)    # 更新最长连续段
            elif not ans[j]:
                path = 0                # 未匹配的位置会切断子串，必须重新计数
        return res
```

专题小结

① 爬楼梯 `dp[i] = dp[i-1] + dp[i-2]`（i ≥ 2）。边界 dp[0] = dp[1] = 1。
② 杨辉三角 `ans[i][j] = ans[i-1][j-1] + ans[i-1][j]`（0 < j < i）。每行两端为 1。
③ 打家劫舍 `dp[i+1] = max(dp[i], dp[i-1] + nums[i])`（i ≥ 1）。dp[k] 对应前 k 间房。
④ 完全平方数 `dp[i] = 1 + min(dp[i-j**2])`，在 1 ≤ j ≤ isqrt(i) 中取最小值。dp[0] = 0。
⑤ 零钱兑换 `dp[i] = 1 + min(dp[i-j])`，在面额 j ≤ i 中取最小值。dp[0] = 0，无可行选择时为无穷大。
⑥ 单词拆分 `dp[i] = OR(dp[j] and s[j:i] == word)`。对词典中满足 j = i - len(word) ≥ 0 的词求或。dp[0] = True。
⑦ 最长递增子序列 `dp[i] = max({1} ∪ {dp[j]+1})`，内层只取 j < i 且 nums[j] < nums[i] 的位置。没有候选时为 1，答案取 max(dp)。
⑧ 乘积最大子数组 令 x = nums[i]，`C = {x, x * max_n[i-1], x * min_n[i-1]}`。转移为 `max_n[i] = max(C)`、`min_n[i] = min(C)`。
⑨ 分割等和子集 令 D(k,j) 表示前 k 个数能否凑出 j，x = nums[k-1]。`D(k,j) = D(k-1,j) or (j >= x and D(k-1,j-x))`。压成一维后按 j 倒序更新。
⑩ 最长有效括号 `j = i - dp[i-1] - 2`。当 s[i-1] 为右括号、j ≥ 0 且 s[j] 为左括号时，`dp[i] = dp[j] + 2 + dp[i-1]`。其余情况为 0，答案取 max(dp)。

核心 先定义状态，再写出边界和转移条件。前缀状态通常取目标位置的值。要求以某处结尾的状态，最后要在所有结尾中取最值。压缩状态时要确认更新顺序仍满足原来的依赖。

16 多维动态规划

16.1 不同路径 LC 62

给定 m 行 n 列的网格，从左上角出发，每一步只能向右或向下。返回到达右下角的不同路径数。

例 `m = 3, n = 7 → 28; m = 1, n = 4 → 1`

```
class Solution:
    def uniquePaths(self, m, n):
        dp = [[0] * n for _ in range(m)]  # dp[i][j] 是到达第 i 行第 j 列的路径数
        for i in range(n):
            dp[0][i] = 1                  # 第一行只能一直向右
        for j in range(m):
            dp[j][0] = 1                  # 第一列只能一直向下
        for i in range(1, m):
            for j in range(1, n):
                # 转移 dp[i][j] = dp[i-1][j] + dp[i][j-1]
                dp[i][j] = dp[i-1][j] + dp[i][j-1]  # 上方与左方的路径数相加
        return dp[m-1][n-1]               # 右下角的路径总数
```

注 状态 `dp[i][j]` 表示从左上角到达坐标 (i, j) 的路径数。起点和边界格都只有一条走法，因此第一行、第一列初始化为 1。

内部格子的最后一步可以从上方或左方进入，转移为 `dp[i][j] = dp[i-1][j] + dp[i][j-1]`。按行从左到右填表时，两项都已算好。

用 `[[0] * n for _ in range(m)]` 为每一行创建独立列表。时间 O(m × n)，额外空间 O(m × n)。

16.2 最小路径和 LC 64

给定非空的非负整数网格 grid，从左上角出发，每步只能向右或向下。返回到达右下角的最小路径和，起点和终点都计入。

例 `grid = [[1,3,1],[1,5,1],[4,2,1]] → 7 (1+3+1+1+1)`

```
class Solution:
    def minPathSum(self, grid):
        m = len(grid)
        n = len(grid[0])
        dp = [[0] * n for _ in range(m)]  # dp[i][j] 是到达 (i, j) 的最小路径和
        a = 0
        for i in range(m):
            dp[i][0] = a + grid[i][0]    # 第一列只能向下走，逐格累加
            a = dp[i][0]
        b = 0
        for j in range(n):
            dp[0][j] = b + grid[0][j]    # 第一行只能向右走，逐格累加
            b = dp[0][j]
        for i in range(1, m):
            for j in range(1, n):
                # 转移 dp[i][j] = min(dp[i-1][j], dp[i][j-1]) + grid[i][j]
                dp[i][j] = min(dp[i-1][j], dp[i][j-1]) + grid[i][j]
        return dp[m-1][n-1]             # 右下角的最小路径和
```

注 状态 `dp[i][j]` 表示到达 (i, j) 的最小路径和。第一行和第一列各自只有一条路径，因此分别沿行和列累加。两次初始化都会把起点设为 grid[0][0]。

内部格子的转移是 `dp[i][j] = min(dp[i-1][j], dp[i][j-1]) + grid[i][j]`。最后一步可以来自上方或左方，取代价更小的一边，再加上当前格子的值。

按行从左到右填表，两项前驱状态都已算好。时间 O(m × n)，额外空间 O(m × n)。

16.3 最长回文子串 LC 5

给定字符串 s，返回其中一个最长回文子串。子串必须连续，长度并列时可以返回任意一个。

例 s = "babad" → "aba"（"bab" 也可）。s = "cbbd" → "bb"。

```
class Solution:
    def longestPalindrome(self, s):
        n = len(s)
        dp = [[False] * n for _ in range(n)]  # dp[i][j] 判断 s[i:j+1] 是否回文
        ans = ""
        for i in range(n-1, -1, -1):          # 先填第 i+1 行，再填第 i 行
            for j in range(i, n):
                # 转移 dp[i][j] = (s[i] == s[j]) and (j-i <= 1 or dp[i+1][j-1])
                dp[i][j] = s[i] == s[j] and (j-i <= 1 or dp[i+1][j-1])
                if dp[i][j] and j-i+1 > len(ans):
                    ans = s[i:j+1]            # 只用更长的回文更新答案
        return ans
```

注 状态 dp[i][j] 表示闭区间 [i, j] 是否为回文。两端相等时，长度为 1 或 2 可以直接确认。更长的区间要看去掉两端后是否为回文。

转移为 dp[i][j] = (s[i] == s[j]) and (j-i <= 1 or dp[i+1][j-1])。右侧依赖第 i+1 行，所以 i 从 n-1 倒着遍历到 0。遍历到第 i 行时，更大的左端点已经处理完毕。

每个区间先按公式给 dp[i][j] 赋布尔值，再用它判断是否更新答案。当 j-i <= 1 时，or 会短路，跳过内部状态的访问。时间 $O(n^2)$，额外空间 $O(n^2)$。

备注 · 双指针中心扩展法

注 本题使用双指针中心扩展。中心 (i, i) 对应奇数长度，(i, i+1) 对应偶数长度。每次比较两端字符，再向外移动一格。易错点 1=<r<=2 有语法错误，正确写法是 1 <= r <= 2。本题用 0 <= l and r < n 分别检查左、右指针的边界。停止扩展时，l 和 r 已越过最后一对匹配的位置，可能越界，也可能指向不相等的字符。有效区间是 [l+1, r)，用 s[l+1:r] 取出，长度为 r-l-1。时间 $O(n^2)$。双指针和下标只用 O(1) 空间，Python 切片会创建新字符串，保存答案最多需要 O(n) 空间。

```
class Solution:
    def longestPalindrome(self, s):
        n = len(s)
        ans = ""
        for i in range(n):
            for l, r in [(i, i), (i, i+1)]:  # 奇数长度与偶数长度的中心
                while 0 <= l and r < n:      # 访问字符前先检查两端下标
                    if s[l] == s[r]:
                        l -= 1               # 当前两端匹配，继续向外扩展
                        r += 1
                    else:
                        break
                if r-l-1 > len(ans):         # 有效区间 [l+1, r) 的长度是 r-l-1
                    ans = s[l+1:r]           # 左端回到 l+1，右端 r 由切片排除
        return ans
```

16.4 最长公共子序列 LC 1143

给定字符串 text1 和 text2，返回最长公共子序列的长度，没有公共字符时返回 0。子序列保留原字符顺序，可以跳过字符。

例 `text1 = "abcde", text2 = "ace" → 3`（公共子序列 "ace"）；`text1 = "abc", text2 = "def" → 0`

```
class Solution:
    def longestCommonSubsequence(self, text1, text2):
        m = len(text1)
        n = len(text2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]  # 空前缀的答案是 0
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if text1[i-1] == text2[j-1]:
                    dp[i][j] = dp[i-1][j-1] + 1  # 转移：接上相同的末尾字符
                else:
                    dp[i][j] = max(dp[i-1][j], dp[i][j-1])  # 转移：分别跳过一个末尾
        return dp[-1][-1]
```

注 状态 `dp[i][j]` 表示 text1[:i] 和 text2[:j] 的最长公共子序列长度。i、j 是前缀长度，末尾字符的下标分别是 i-1 和 j-1。任一前缀为空时，长度为 0。

末尾字符相同，接在两个更短前缀的公共子序列后面：`dp[i][j] = dp[i-1][j-1] + 1`。末尾不同，至少要跳过其中一个：`dp[i][j] = max(dp[i-1][j], dp[i][j-1])`。

从左到右逐行填表，上方、左方和左上方都已算好。时间 O(m × n)，额外空间 O(m × n)。

16.5 编辑距离 LC 72

给定字符串 word1 和 word2，每次可以在 word1 中插入、删除或替换一个字符。返回把 word1 变为 word2 所需的最少操作次数。

例 word1 = "horse", word2 = "ros" → 3 (horse → rorse → rose → ros)；word1 = "", word2 = "abc" → 3

```
class Solution:
    def minDistance(self, word1, word2):
        m = len(word1)
        n = len(word2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            dp[i][0] = i                            # 删掉 word1 前缀的全部字符
        for j in range(1, n + 1):
            dp[0][j] = j                            # 从空串插入 word2 的前缀
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if word1[i-1] == word2[j-1]:
                    dp[i][j] = dp[i-1][j-1]         # 转移：保留相同字符，代价为 0
                else:
                    # 转移 dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-
                    dp[i][j] = min(dp[i-1][j] + 1,      # 删除
                                   dp[i][j-1] + 1,      # 插入
                                   dp[i-1][j-1] + 1)    # 替换
        return dp[-1][-1]
```

注 状态 dp[i][j] 表示把 word1[:i] 变为 word2[:j] 的最少操作数。第一列 dp[i][0] = i，第一行 dp[0][j] = j，分别对应全部删除和全部插入。
末尾相同：dp[i][j] = dp[i-1][j-1]。末尾不同：dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])，三项依次对应删除、插入和替换。
按 i、j 从小到大填表，当前状态所需的三个相邻状态都已算好。时间 O(m × n)，额外空间 O(m × n)。

专题小结

① 不同路径 dp[i][j] = dp[i-1][j] + dp[i][j-1]。第一行和第一列都为 1。
② 最小路径和 dp[i][j] = min(dp[i-1][j], dp[i][j-1]) + grid[i][j]。边界沿第一行、第一列累加。
③ 最长回文子串 dp[i][j] = (s[i] == s[j]) and (j-i <= 1 or dp[i+1][j-1])。i 倒序，先算内层依赖。
④ 最长公共子序列 末尾相同：dp[i][j] = dp[i-1][j-1] + 1；否则：dp[i][j] = max(dp[i-1][j], dp[i][j-1])。空前缀为 0。
⑤ 编辑距离 末尾相同：dp[i][j] = dp[i-1][j-1]；否则：dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])。空前缀边界为 dp[i][0] = i、dp[0][j] = j。

核心 先分清下标表示网格坐标、区间端点还是前缀长度，再根据依赖决定填表顺序。两个前缀的题常用额外一行一列表示空前缀，取末尾字符时用 i-1 和 j-1。

17 技巧

5/5

17.1 只出现一次的数字 LC 136

非空整数数组 nums 中，只有一个数字出现一次，其余数字都出现两次；返回这个只出现一次的数字，要求线性时间、常数额外空间。

例 nums = [4,1,2,1,2] → 4（两个 1 和两个 2 抵消，只剩 4）

```
class Solution:
    def singleNumber(self, nums):
        x = 0                           # 异或的初值，0 与任何数异或仍是那个数
        for num in nums:
            x = x ^ num                 # 相同数字异或为 0，成对出现的数会抵消
        return x                        # 最后留下的就是只出现一次的数字
```

注 异或规则：`a ^ a == 0`、`a ^ 0 == a`；异或满足交换律和结合律，所以重复的数即使不相邻也能抵消。

`^` 是整数按位异或，乘方用 `**`。本题返回单个整数，不返回列表，也不修改 nums。

17.2 多数元素 LC 169

非空数组 nums 中有一个元素出现次数严格大于 len(nums) / 2，返回它；题目保证这个多数元素一定存在。

例 nums = [2,2,1,1,1,2,2] → 2（2 出现 4 次，超过总数 7 的一半）

```
class Solution:
    def majorityElement(self, nums):
        ans = nums[0]                   # 当前候选数，题目保证数组非空
        hp = 1                          # 候选的抵消计数，不是实际出现次数
        for i in range(1, len(nums)):
            if nums[i] == ans:
                hp += 1                 # 遇到相同的数，计数加一
            else:
                hp -= 1                 # 遇到不同的数，抵消一次
            if hp == 0:
                ans, hp = nums[i], 1    # 归零就换当前数为候选，重新计数
        return ans                      # 有严格多数的保证，最后候选就是答案
```

注 `hp` 是抵消计数，不是候选的真实出现次数；若题目不保证多数元素存在，还需统计候选次数并验证。

17.3 颜色分类 LC 75

nums 只含 0、1、2，分别表示三种颜色；原地把相同颜色排在一起，并按 0、1、2 的顺序排列，不返回数组。

例 nums = [2,0,2,1,1,0] → nums 变为 [0,0,1,1,2,2]（返回值是 None）

```
class Solution:
    def sortColors(self, nums):
        p0 = p1 = 0                     # p0 是 0 区右边界，p1 是 0/1 区右边界
        for i, x in enumerate(nums):     # x 保存当前位置写入前的原值
            nums[i] = 2                 # 先把新位置归到 2 区
            if x <= 1:
                nums[p1] = 1            # 原值为 0 或 1，先扩大 0/1 区
                p1 += 1
            if x == 0:
                nums[p0] = 0            # 原值为 0，再把 0 区扩大一位
                p0 += 1
```

注 循环开始时，[0, p0) 全是 0，[p0, p1) 全是 1，[p1, i) 全是 2。两个 if 不能改成 if / elif：遇到 0 时，两处都要执行。

17.4 下一个排列 LC 31

将 nums 原地重排为字典序紧邻的下一个排列；若已经是最大排列，就变为升序的最小排列。允许重复元素，不返回数组。

例 nums = [1,2,3] → nums 变为 [1,3,2]；nums = [3,2,1] → [1,2,3]（均返回 None）

```
class Solution:
    def nextPermutation(self, nums):
        n = len(nums)
        i = n - 2                               # 从倒数第二位找能增大的位置
        while i >= 0 and nums[i] >= nums[i + 1]:
            i -= 1                              # 右边这段不升序，继续向左找
        if i >= 0:                              # 找到后，先把这一位换大
            j = n - 1                           # 从右找后缀中最小的、更大的数
            while nums[j] <= nums[i]:
                j -= 1
            nums[i], nums[j] = nums[j], nums[i]  # 换成刚好更大的数
        left, right = i + 1, n - 1              # i 为 -1 时，范围就是整个数组
        while left < right:                     # 原地反转后缀，使它升序、增幅最小
            nums[left], nums[right] = nums[right], nums[left]
            left += 1
            right -= 1
```

17.5 寻找重复数 LC 287

nums 长度为 n + 1，所有数都在 1..n 内，只有一种数字出现至少两次，返回这个重复数；不能修改 nums，额外空间要求 O(1)。

例 nums = [1,3,4,2,2] → 2；nums = [3,3,3,3,3] → 3（重复次数可以超过两次）

```
class Solution:
    def findDuplicate(self, nums):
        slow = fast = 0                    # 从下标 0 出发，nums[i] 是下一位置
        while True:
            slow = nums[slow]              # 慢指针每次走一步
            fast = nums[nums[fast]]         # 快指针每次走两步
            if fast == slow:
                break                      # 先找到环内相遇点，未必是环入口
        head = 0                           # 一个从起点走，一个从相遇点走
        while slow != head:
            slow = nums[slow]              # 两个指针同步，每次各走一步
            head = nums[head]
        return slow                        # 再次相遇的位置就是环入口和重复数
```

专题小结

① 只出现一次：利用相同数字异或为 0，让成对的数抵消。
② 多数元素：维护一个候选和抵消计数；题目保证严格多数存在，最后的候选就是答案。
③ 颜色分类：用两个边界维护 0 区、1 区和 2 区，边扫描边放回正确区域。
④ 下一个排列：从右边找能增大的位置，换上右侧刚好更大的数，再反转后缀，让增幅尽量小。
⑤ 寻找重复数：把下标到 nums[下标] 当成指针，先找环内相遇点，再从 0 和相遇点同步走，找出环入口。

核心 这些技巧依赖各自的题目条件：异或需要其余数成对；候选抵消依赖严格多数的存在；颜色分类只处理三种取值；下一个排列利用后缀的不升序；快慢指针利用长度和取值范围形成环。先确认条件，再使用对应方法，不能把它们当成同一种通用规则。

A 库函数速查

按模块分组，组名后面标了要 import 什么。语法末尾的小徽章是行为标记：**原地**＝修改原对象，返回 None；**返回**＝取得结果或迭代器；取用迭代器会推进位置；**原地+返回**＝可能修改原对象，同时给出结果。赋值、作用域、流程控制、规则徽章表示语法用途，不是函数返回值。示例结果若是异常名称，表示运行该表达式会抛出该异常。先按组内提示导入。带准备步骤的例子先运行「程序」，再求值「查看结果」中的表达式。箭头右侧是预期结果。输入输出在第 0 专题。常用组含相关基础方法；其余模块随正文需要显示。

排序 （不用 import）

默认升序，reverse=True 降序；两种排序都稳定，key 相同的元素保持原有先后。只能比较彼此可排序的元素。
语法行展示常用参数；变量名是占位，实际调用看下一行示例。

`sorted(iterable, key=None, reverse=False)` 返回

返回排序后的新列表，不修改原容器；key、reverse 用关键字传入。

```
sorted([3, 1, 2]) -> [1, 2, 3]
```

`a.sort(key=None, reverse=False)` 原地

列表原地排序，返回 None；不要写 a = a.sort()。

程序

```
a = [3, 1, 2]
a.sort()
```

查看结果 `a` → `[1, 2, 3]`

`key=lambda x: x[0]` 参数

下标从 0 起：x[0] 是第一个字段，x[1] 是第二个。元组多关键字依次比较；数字前加负号可让该字段降序。

```
sorted([(1, 2), (1, 3)], key=lambda x: (x[0], -x[1])) -> [(1, 3), (1, 2)]
sorted(['bb', 'a'], key=len) -> ['a', 'bb']
```

查找下标 （str / list 不用 import）

find / rfind 用于字符串，找不到返回 -1；str.index / list.index 找不到抛 ValueError。
可选起止下标是左闭右开，作为位置参数传入；有序数组找插入位置可用 bisect。

`s.find(sub)` 返回

返回子串第一次出现的下标；可写 s.find(sub, start, end)。

```
'abcabc'.find('b') -> 1
'abcabc'.find('z') -> -1
```

`s.index(sub) / a.index(x)` 返回

返回第一次出现的下标；可加起止下标。找不到抛 ValueError。

```
[3, 1, 3].index(3) -> 0
[1].index(9) -> ValueError
```

`s.rfind(sub)` 返回

返回子串最后一次出现的下标；可写 s.rfind(sub, start, end)。

```
'abcabc'.rfind('b') -> 4
```

堆 heapq (import heapq)

先 import heapq。为兼容较旧的考试环境，这里用小根堆；大根堆可存负数，弹出时取反。Python 3.14 起另有公开的大根堆接口。
堆只保证堆顶最小，不保证整表有序。基础堆操作没有 key 参数；元组逐项比较，后续字段也必须可比较，或加入唯一计数器避免比较对象。

heapq.heapify(h) 原地

原地建小根堆，O(n)，返回 None。

程序
```
h = [3, 1, 2]
heapq.heapify(h)
```

查看结果 `h[0]` → **1**

heapq.heappop(h) 原地+返回

移除并返回最小值，O(log n)；空堆抛 IndexError。

程序
```
h = [1, 3, 2]
```

查看结果 `heapq.heappop(h)` → **1**

heapq.heappush(h, x) 原地

压入一个元素，O(log n)，返回 None。

程序
```
h = [1, 3, 2]
heapq.heappush(h, 0)
```

查看结果 `(h[0], len(h))` → **(0, 4)**

heapq.heapreplace(h, x) 原地+返回

移除并返回旧堆顶，再压入 x；堆不能为空。

程序
```
h = [1, 3, 2]
```

查看结果 `heapq.heapreplace(h, 5)` → **1**

二分 bisect (import bisect)

先 import bisect。列表必须已按升序排列；返回插入位置，可能等于 len(a)，不代表目标存在。

bisect.**bisect_left**(a, x) 返回

返回第一个 >= x 的位置；可用 lo、hi 限定左闭右开区间。判存在还需比较值。

`bisect.bisect_left([1, 3, 3, 5], 3) -> 1`

程序

```
a = [1, 3, 5]
i = bisect.bisect_left(a, 4)
```

查看结果 `i < len(a) and a[i] == 4` → **False**

bisect.**bisect_right**(a, x) 返回

返回第一个 > x 的位置；右边界减左边界就是有序数组中 x 的个数。

`bisect.bisect_right([1, 3, 3, 5], 3) -> 3`

队列与计数 collections (import collections)

先 import collections。q 是 collections.deque 的实例；字典工厂用 collections.defaultdict。
两端追加、弹出为 O(1)；中间下标访问可能为 O(n)。空队列弹出会抛 IndexError。

q.**appendleft**(x) 原地

左端追加，返回 None。

程序

```
q = collections.deque([1, 2])
q.appendleft(0)
```

查看结果 `list(q)` → **[0, 1, 2]**

collections.**Counter**(iterable) 返回

先 import collections，也可 from collections import Counter。返回计数对象。用 c[key] 查缺失的键时返回 0，不会插入该键。Python 3.10 起可用 >= 逐项比较计数，缺项按 0 处理。

`collections.Counter('aba')['a'] -> 2`

`collections.Counter('aba')['x'] -> 0`

`collections.Counter('aba') >= collections.Counter('aa') -> True`

collections.**defaultdict**(default_factory) 返回

缺键时，d[key] 调工厂生成默认值并存入字典；get() 不触发工厂。工厂可用 int、list、set。

程序

```
d = collections.defaultdict(int)
d['x'] += 1
```

查看结果 `dict(d)` → **{'x': 1}**

程序

```
d = collections.defaultdict(list)
d['k'].append(1)
```

查看结果 `dict(d)` → **{'k': [1]}**

collections.deque(iterable=(), maxlen=None) 返回

构造双端队列。maxlen 限长时，追加时若已满，append / appendleft 会挤掉另一端元素。

程序

```
q = collections.deque([1, 2])
```

查看结果 `list(q)` → **[1, 2]**

程序

```
q = collections.deque([1, 2], maxlen=2)
q.append(3)
```

查看结果 `list(q)` → **[2, 3]**

q.append(x) 原地

右端追加，返回 None。

程序

```
q = collections.deque([1, 2])
q.append(3)
```

查看结果 `list(q)` → **[1, 2, 3]**

q.pop() 原地+返回

移除并返回右端元素。

程序

```
q = collections.deque([1, 2])
```

查看结果 `q.pop()` → **2**

q.popleft() 原地+返回

移除并返回左端元素；BFS 常用。

程序

```
q = collections.deque([1, 2])
```

查看结果 `q.popleft()` → **1**

列表 list (lst.xxx)

a 是列表。原地方法通常返回 None；pop 既修改列表又返回元素。copy 只做浅拷贝，嵌套对象仍共享。

a.append(x) 原地

尾部追加一个元素。

程序

```
a = [1]
a.append([2, 3])
```

查看结果 a → **[1, [2, 3]]**

a.clear() 原地

移除全部元素，返回 None。

程序

```
a = [1, 2]
a.clear()
```

查看结果 a → **[]**

a.copy() 返回

复制外层列表，内部可变对象仍共享。

程序

```
a = [1, 2]
```

查看结果 a.copy() → **[1, 2]**

程序

```
a = [[1]]
b = a.copy()
b[0].append(2)
```

查看结果 a → **[[1, 2]]**

a.extend(iterable) 原地

把可迭代对象的元素逐个追加，区别于 append 整体追加。

程序

```
a = [1]
a.extend([2, 3])
```

查看结果 a → **[1, 2, 3]**

a.insert(i, x) 原地

在下标 i 前插入，O(n)。

程序

```
a = [1, 3]
a.insert(1, 2)
```

查看结果 a → **[1, 2, 3]**

a.pop() / a.pop(i) 原地+返回

移除并返回下标 i 的元素；默认末尾，i 用位置参数传入。越界或空列表抛 IndexError。

程序

```
a = [1, 2, 3]
```

查看结果 a.pop() → 3

a.remove(x) 原地

移除第一个等于 x 的元素；不存在抛 ValueError。

程序

```
a = [1, 2, 1]
a.remove(1)
```

查看结果 a → [2, 1]

a.reverse() 原地

原地反转，返回 None；需要新列表可写 a[::-1]。

程序

```
a = [1, 2, 3]
a.reverse()
```

查看结果 a → [3, 2, 1]

a[start:stop:step] 返回

stop 不包含在内。读取列表切片会创建新列表。a[:] = new 修改原列表，a = new 则重新绑定名字。

`[0, 1, 2, 3][1:3] -> [1, 2]`

程序

```
a = [1, 2]
b = a
a[:] = [3]
```

查看结果 b → [3]

列表推导式 `[表达式 for 变量 in 可迭代对象 if 条件]` 返回

无需 import。按遍历顺序计算表达式，得到一个新列表。末尾的 if 条件可以省略。

记忆句 `[表达式 for 变量 in 可迭代对象 if 条件]`

遍历

程序
```
nums = [1, 2, 3]
```

查看结果 `[i for i in nums]` → **`[1, 2, 3]`**

筛选

程序
```
nums = [1, 2, 3]
```

查看结果 `[i for i in nums if i > 1]` → **`[2, 3]`**

转换

程序
```
nums = [1, 2, 3]
```

查看结果 `[i * 2 for i in nums]` → **`[2, 4, 6]`**

筛选 + 转换

程序
```
nums = [1, 2, 3]
```

查看结果 `[i * 2 for i in nums if i > 1]` → **`[4, 6]`**

字典 dict (d.xxx)

d 是字典。in 默认查键；查键和按键取值通常为平均 O(1)。遍历 items / keys / values 时不要增删字典条目。

d[key] 返回

按键取值；缺键抛 KeyError。

`{'a': 1}['a'] -> 1`

d[key] = val 原地

键存在就覆盖，不存在就新建。

程序
```
d = {}
d['a'] = 1
```

查看结果 `d` → **`{'a': 1}`**

key in d 返回

判断键是否存在，不查值。

`'a' in {'a': 1} -> True`

d.get(key) / d.get(key, default) 返回

按键取值，缺键返回默认值，不自动插入。默认值用位置参数传入。

`{'a': 1}.get('b', 0) -> 0`

d.pop(key) / d.pop(key, default) 原地+返回

移除并返回键对应的值；default 可省，缺键且无默认值抛 KeyError。

程序

```
d = {'a': 1}
```

查看结果 `d.pop('a')` → **1**

`{}.pop('x', 0) -> 0`

d.items() 返回

返回键值对视图；需要列表时再 list()。

`list({'a': 1}.items()) -> [('a', 1)]`

d.keys() 返回

返回键视图；直接 for key in d 也会遍历键。

`list({'a': 1}.keys()) -> ['a']`

d.setdefault(key) / d.setdefault(key, default) 原地+返回

存在就返回原值；缺键则插入并返回 default，省略时插入 None。参数按位置传入。

程序

```
d = {}
d.setdefault('x', []).append(1)
```

查看结果 `d` → **{'x': [1]}**

d.update(other) 原地

用另一个映射或键值对序列更新字典；同键覆盖，返回 None。

程序

```
d = {'a': 1}
d.update({'a': 2, 'b': 3})
```

查看结果 `d` → **{'a': 2, 'b': 3}**

d.values() 返回

返回值视图。

`list({'a': 1}.values()) -> [1]`

字符串 str (s.xxx)

不修改原字符串；split 返回 list，find / count 返回 int，join / strip 等返回 str，is... 方法返回 bool。
字符判断方法识别 Unicode，不限于英文字母或 ASCII 数字；下列 is... 判断中，除 isascii 外，空串都返回 False。

`s.split(sep=None, maxsplit=-1)` 返回

省略 sep 按任意空白切分，并丢掉首尾空白及空分片。指定 sep 按该子串切分，连续分隔符保留空分片；maxsplit 限制切分次数。

`' a b '.split() -> ['a', 'b']`

`'a,b,,c'.split(',') -> ['a', 'b', '', 'c']`

`'a,b,c'.split(',', 1) -> ['a', 'b,c']`

`sep.join(iterable)` 返回

用 sep 连接字符串；元素必须全是 str。join 不会自动把整数转成字符串。

`','.join(['a', 'b', 'c']) -> 'a,b,c'`

`' '.join(map(str, [1, 2, 3])) -> '1 2 3'`

`data.decode('utf-8')` 返回

按指定编码把 bytes 解码为 str。编码不匹配时默认抛 UnicodeDecodeError。

`b'abc'.decode('utf-8') -> 'abc'`

`s.count(sub)` 返回

统计不重叠的子串出现次数；不是滑动重叠计数。

`'aaaa'.count('aa') -> 2`

`s.endswith(suffix)` 返回

判断后缀；suffix 可是字符串或字符串元组。

`'hello.py'.endswith(('.py', '.txt')) -> True`

`s.isalnum()` 返回

非空且全是字母或数值字符；下划线不算。

`'a1'.isalnum() -> True`

`'a_1'.isalnum() -> False`

`s.isalpha()` 返回

非空且全是 Unicode 字母，中文也可通过；不代表仅 a-z / A-Z。

`'中文'.isalpha() -> True`

`'a1'.isalpha() -> False`

`s.isascii()` 返回

是否全部为 ASCII 字符；空串也返回 True。

`'abc123'.isascii() -> True`

`'中'.isascii() -> False`

`s.isdecimal()` 返回

非空且全是十进制数字字符；比 isdigit 更窄。只接受 ASCII 数字可同时判断 isascii()。

`'²'.isdecimal() -> False`

程序

```
s = '123'
```

查看结果 `s.isascii() and s.isdecimal()` → **True**

`s.isdigit()` 返回

非空且全是数字字符，包括部分上标数字。返回 True 也不保证 int() 能转换。

`'123'.isdigit() -> True`

`'²'.isdigit() -> True`

`'-1'.isdigit() -> False`

`'1.5'.isdigit() -> False`

`int('²') -> ValueError`

`s.islower()` 返回

至少有一个区分大小写的字符，且这些字符全为小写；数字不影响判断。

`'abc1'.islower() -> True`

`'123'.islower() -> False`

`s.isnumeric()` 返回

非空且全是 Unicode 数值字符；范围比 isdigit 更宽，包含部分分数。True 不保证 int() 能转换。

`'½'.isnumeric() -> True`

`s.isspace()` 返回

非空且全是空白字符，包含空格、Tab、换行。

`' \t'.isspace() -> True`

`''.isspace() -> False`

`s.isupper()` 返回

至少有一个区分大小写的字符，且这些字符全为大写。

`'ABC1'.isupper() -> True`

`s.lower()` 返回

返回转小写后的字符串。

`'AbC'.lower() -> 'abc'`

`s.lstrip()` / `s.lstrip(chars)` 返回

只去掉左端指定字符或空白；保留行内和右端字符。

`' a '.lstrip() -> 'a '`

`s.replace(old, new)` / `s.replace(old, new, count)` 返回

返回替换后的字符串；count 限制替换次数，省略或传 -1 则全部替换。为兼容旧版 Python，把 count 作为第三个位置参数。

`'a-b-c'.replace('-', '+', 1) -> 'a+b-c'`

`s.rstrip()` / `s.rstrip(chars)` 返回

只去掉右端指定字符或空白；chars 是字符集合。若只应去掉一组 LF 或 CRLF，见输入⑫的去行尾模板。

`'a\n'.rstrip('\n') -> 'a'`

`s.startswith(prefix)` 返回

判断前缀；prefix 可是字符串或字符串元组。

`'hello.py'.startswith('hello') -> True`

`s.strip()` / `s.strip(chars)` 返回

默认去掉两端空白；chars 指字符集合，不是完整子串。

`' a '.strip() -> 'a'`

`'xyaxy'.strip('xy') -> 'a'`

`' a '.strip(chars=' ') -> TypeError`

`s.upper()` 返回

返回转大写后的字符串。

`'AbC'.upper() -> 'ABC'`

集合 set (st.xxx)

st 是集合，元素必须可哈希；集合无序，不能用下标访问。空集合写 set()，{} 是空字典。成员判断平均 O(1)。

`st.add(x)` 原地

加入一个元素；已有元素不重复添加，返回 None。

程序

```
st = {1}
st.add(2)
```

查看结果 `st` → **{1, 2}**

`st.discard(x)` 原地

移除元素，不存在也不报错，返回 None。

程序

```
st = {1, 2}
st.discard(9)
```

查看结果 `st` → **{1, 2}**

`st.remove(x)` 原地

移除元素，不存在抛 KeyError，返回 None。

程序

```
st = {1, 2}
st.remove(1)
```

查看结果 `st` → **{2}**

`st.pop()` 原地+返回

移除并返回任意一个元素；不是按顺序弹出。空集合抛 KeyError。

程序

```
st = {7}
```

查看结果 `st.pop()` → **7**

`a & b / a | b / a - b / a ^ b` 返回

分别返回交集、并集、差集、对称差集，不修改 a、b。

```
{1, 2} & {2, 3} -> {2}
{1, 2} | {2, 3} -> {1, 2, 3}
{1, 2} - {2, 3} -> {1}
{1, 2} ^ {2, 3} -> {1, 3}
```

内置函数 (不用 import)

无需 import。返回迭代器的函数不会自动产生列表，需要保存或显示所有元素时用 list()。迭代器通常只能消耗一次。

`abs(x)` 返回

返回绝对值。

```
abs(-3) -> 3
```

`all(iterable)` 返回

所有元素为真则 True，短路；空输入为 True。

```
all([True, False]) -> False
all([]) -> True
```

`any(iterable)` 返回

任意元素为真则 True，短路；空输入为 False。

```
any([False, True]) -> True
any([]) -> False
```

`chr(i)` 返回

Unicode 码点转单个字符；越界抛 ValueError。

```
chr(97) -> 'a'
```

`dict() / dict(pairs)` 返回

构造字典；pairs 中每项是一个键值对。

```
dict(zip('ab', [1, 2])) -> {'a': 1, 'b': 2}
```

`divmod(a, b)` 返回

返回 (a // b, a % b)；整数除数不能为 0。

```
divmod(7, 3) -> (2, 1)
```

`enumerate(iterable, start=0)` 返回

返回逐项产生 (编号, 元素) 的迭代器。

```
list(enumerate('ab', 1)) -> [(1, 'a'), (2, 'b')]
```

`filter(func, iterable)` 返回

返回仅保留判断为真的元素的迭代器；func 为 None 时按元素自身真假筛选。

```
list(filter(lambda x: x > 0, [-1, 0, 2])) -> [2]
```

`float(x)` 返回

转换为浮点数；非法数字文本抛 ValueError。float('inf') 和 float('-inf') 分别返回正无穷和负无穷，可作为边界或不可达状态的初值。

`float('1.5') -> 1.5`

`float('inf') > 10**308 -> True`

`float('-inf') < -100 -> True`

`int(x) / int(text, base)` 返回

数字转整数向零截断；指定 base 时，输入须为 str、bytes 或 bytearray。

`int('10', 2) -> 2`

`int(-3.9) -> -3`

`iter(iterable)` 返回

创建迭代器；next() 每次消耗一个元素。

程序

```
it = iter([4, 5])
```

查看结果 `next(it)` → **4**

`len(obj)` 返回

返回内置容器元素数或字符串字符数；不是迭代器通用接口。

`len('abc') -> 3`

`list(iterable=())` 返回

构造列表；将迭代器的剩余元素物化。

`list(range(3)) -> [0, 1, 2]`

`map(func, iterable)` 返回

返回逐个应用 func 的迭代器；多输入时到最短输入结束。

`list(map(int, ['1', '2'])) -> [1, 2]`

`max(iterable, key=None) / max(a, b, ...)` 返回

返回最大元素；空可迭代对象需给 default，否则 ValueError。default 只适用于单个 iterable 的形式。

`max([1, 3, 2]) -> 3`

`max([], default=0) -> 0`

`max(2, 5) -> 5`

`min(iterable, key=None) / min(a, b, ...)` 返回

返回最小元素；空输入、default 的规则同 max。

`min([1, 3, 2]) -> 1`

`next(iterator) / next(iterator, default)` 返回

取下一项；default 可省，按位置传入，耗尽且无默认值抛 StopIteration。

`next(iter([]), None) -> None`

`ord(c)` 返回

单个 Unicode 字符转码点；字符串长度必须为 1。

`ord('a') -> 97`

pow(base, exp, mod=None) 返回

两参数时与 base ** exp 同义；三个整数参数可直接计算模幂，mod 不能为 0。

`pow(2, 3) -> 8`

`pow(2, 10, 7) -> 2`

---

range(stop) / range(start, stop, step=1) 返回

整数范围，左闭右开；step 可负但不能为 0。方向不匹配时为空。

`list(range(3)) -> [0, 1, 2]`

`list(range(2, -1, -1)) -> [2, 1, 0]`

---

reversed(seq) 返回

返回逆序迭代器，不修改原序列。

`list(reversed([1, 2, 3])) -> [3, 2, 1]`

---

round(number, ndigits=None) 返回

返回舍入后的数值，不保证显示固定小数位。恰好中点时取偶数；浮点表示可能影响十进制预期。

`round(2.5) -> 2`

`round(3.5) -> 4`

---

set(iterable=()) 返回

构造集合并去重；元素须可哈希。

`set([1, 1, 2]) -> {1, 2}`

---

str(obj) 返回

转换成字符串；容器转换不是按指定分隔符连接，要连接元素用 join。

`str(12) + '3' -> '123'`

---

sum(iterable, start=0) 返回

累加数值；不用于连接字符串。二维列表展平优先用推导式，避免反复复制。

`sum([1, 2, 3]) -> 6`

程序

```
g = [[1, 2], [3]]
```

查看结果 `[x for row in g for x in row]` → **`[1, 2, 3]`**

---

zip(*iterables) 返回

返回按位置配对的迭代器，默认到最短输入结束；转置要求矩阵各行等长。

`list(zip('ab', [1, 2])) -> [('a', 1), ('b', 2)]`

(表达式 for 变量 in 可迭代对象 if 条件) 返回

生成器按需计算，用 next() 或循环消费，一般只能遍历一遍。与列表推导式相比，不会一次建好结果列表。

程序

```
g = (x * x for x in [1, 2, 3])
```

查看结果 `(next(g), list(g), list(g))` → **`(1, [4, 9], [])`**

递归深度 (import sys)

先 import sys。只在确需更深递归时调整；过高的上限不保证能安全递归到该深度，深链可改用显式栈。

`sys.getrecursionlimit()` 返回

返回当前解释器的递归深度上限。先读出再决定是否提高。

`sys.getrecursionlimit() > 0 -> True`

`sys.setrecursionlimit(n)` 设置

设置解释器递归深度上限，返回 None；不是预分配或保证可用的栈空间。

程序

```
old = sys.getrecursionlimit()
sys.setrecursionlimit(max(old, 2000))
changed = sys.getrecursionlimit()
sys.setrecursionlimit(old)
```

查看结果 `changed >= 2000` → **True**

数学 math (import math)

math.xxx 先 import math。整数平方根用 isqrt，普通 sqrt 返回浮点数；乘方 ** 见运算符组。

`math.ceil(x)` 返回

向正无穷取整。

`math.ceil(-1.2) -> -1`

`math.floor(x)` 返回

向负无穷取整；区别于 int() 的向零截断。

`math.floor(-1.2) -> -2`

`math.gcd(a, b)` 返回

返回非负最大公约数。

`math.gcd(12, 18) -> 6`

`math.inf` 属性

正无穷；负无穷写 -math.inf，常用作边界初值。

`math.inf > 10**308 -> True`

`math.isqrt(n)` 返回

返回非负整数 n 的平方根向下取整值，结果为 int；负数抛 ValueError。枚举 j**2 <= n，可写 range(1, math.isqrt(n) + 1)。

`math.isqrt(10) -> 3`

`math.isqrt(16) -> 4`

`math.sqrt(x)` 返回

返回平方根浮点值；负数在实数域抛 ValueError。

`math.sqrt(9) -> 3.0`

运算符 (不用 import)

以下语法行是写法示意，等宽示例可直接运行。算术、比较、逻辑和位运算均不用 import。

a % b 返回

取余；非零余数与除数同号。整数满足 a == (a // b) * b + a % b；除数不能为 0。

```
7 % 2 -> 1
-7 % 3 -> 2
7 % -3 -> -2
```

---

a ** b（平方：j ** 2） 返回

乘方，不用 import；j ** 2 是平方。^ 是按位异或，不能当作平方或乘方。

```
3 ** 2 -> 9
2 ** 3 -> 8
3 ^ 2 -> 1
```

---

a + b、a - b、a * b 返回

数值加、减、乘。

```
2 + 3 -> 5
7 - 4 -> 3
3 * 4 -> 12
```

---

a / b 返回

真除法；两个整数相除也返回 float。除数不能为 0。

```
7 / 2 -> 3.5
6 / 2 -> 3.0
```

---

a // b 返回

向负无穷取整的除法，不是向零截断。整数输入得到 int；有浮点输入时结果为 float。除数不能为 0。

```
7 // 3 -> 2
-7 // 3 -> -3
```

---

a and b 返回

逻辑与，短路；a 为假时直接返回 a，否则求值并返回 b，不保证返回 bool。

```
0 and 5 -> 0
2 and 5 -> 5
```

---

not a 返回

对对象的真假取反，返回 bool；0、空容器、None 为假。

```
not [] -> True
not 3 -> False
```

---

a or b 返回

逻辑或，短路；a 为真时直接返回 a，否则求值并返回 b。左侧为真时不会求值右侧，不保证返回 bool。

```
0 or 5 -> 5
2 or 5 -> 2
```

括号、乘方、乘除取余、加减 规则

常用顺序按此排列，复杂表达式用括号说明意图。** 右结合；负号与乘方一起使用时注意括号。

```
2 + 3 * 4 -> 14
(2 + 3) * 4 -> 20
2 ** 3 ** 2 -> 512
-2 ** 2 -> -4
(-2) ** 2 -> 4
```

a & b、a | b、a ^ b、~a 返回

这里是整数的按位与、或、异或、取反；不是逻辑 and / or。集合的 &、| 另见集合组。

```
5 & 3 -> 1
5 | 3 -> 7
5 ^ 3 -> 6
~5 -> -6
```

x in obj、x not in obj 返回

判断成员是否存在；字典默认查键，字符串查子串。

```
2 in [1, 2] -> True
'a' in {'a': 1} -> True
```

a if condition else b 返回

按条件选择一个结果，只求值选中的分支。

```
'yes' if 3 > 2 else 'no' -> 'yes'
```

==、!=、<、<=、>、>= 返回

内置数值和字符串的比较返回 bool；支持连续比较。赋值 = 不等于比较 ==。

```
3 == 3 -> True
3 != 4 -> True
1 < 2 <= 2 -> True
```

a << k、a >> k 返回

整数左移、右移，k 不能为负。左移相当于乘 2**k，右移相当于 // 2**k。

```
3 << 2 -> 12
12 >> 1 -> 6
-3 >> 1 -> -2
```

=、+=、-=、*=、/=、//=、%= 赋值

给变量赋值会绑定名字，也可以给下标或属性赋值；增强赋值的行为取决于类型。整数 += 重新绑定，列表 += 可原地扩展并影响别名。

程序

```
x = 2
x += 3
```

查看结果 `x` → **`5`**

程序

```
a = [1]
b = a
a += [2]
```

查看结果 `b` → **`[1, 2]`**

a is b、a is not b 返回

判断是否同一对象，不等于数值或内容相等；判断空值常写 x is None。

程序

```
a = [1]
b = [1]
```

查看结果 `a == b` → **`True`**

程序

```
a = [1]
b = [1]
```

查看结果 `a is b` → **`False`**

作用域（不用 import）

以下是作用域语法，不是库函数，不用 import。代码例子中的缩进必须保留；单独读取外层变量不需要声明。

global name 作用域

在函数中声明名字属于模块全局作用域；与 nonlocal 指向外层函数不同。

程序

```
count = 0
def add():
    global count
    count += 1
add()
```

查看结果 `count` → **`1`**

`nonlocal name` 作用域

在嵌套函数中，重新绑定最近外层函数已经存在的变量。不能用来指向模块全局，也不能凭空创建外层变量；声明应在使用该名字之前。

程序

```
def outer():
    flag = False
    def inner():
        nonlocal flag
        flag = True
    inner()
    return flag
```

查看结果 `outer()` → **`True`**

`path.append(x)`、`path.pop()` 作用域

修改外层列表对象本身不需 nonlocal；对 path 重新赋值才是重新绑定名字。

程序

```
def outer():
    path = []
    def add():
        path.append(1)
    add()
    return path
```

查看结果 `outer()` → **`[1]`**

函数内赋值 作用域

函数内重新绑定名字默认创建局部变量；外层同名变量不会因此改变。

程序

```
x = 7
def f():
    x = 2
    return x
```

查看结果 `(f(), x)` → **`(2, 7)`**

流程控制 (不用 import)

本组收录条件分支、循环和退出语句。用缩进确定语句所属的代码块。

`if 条件: … elif 条件: … else: …` 流程控制

按顺序判断条件，只执行首个为真的分支。前面的条件全为假时执行 else。elif 和 else 都可省略。

程序

```
x = 0
if x < 0:
    label = '负数'
elif x == 0:
    label = '零'
else:
    label = '正数'
```

查看结果 `label` → `'零'`

`for 变量 in 可迭代对象: …` 流程控制

依次取出元素并执行循环体。遍历索引可用 range，同时取下标和值可用 enumerate。

程序

```
total = 0
for x in [1, 2, 3]:
    total += x
```

查看结果 `total` → `6`

`while 条件: …` 流程控制

每轮开始前判断条件。条件为真才执行循环体，初始为假时执行零次。while True 常配合 break 或 return 结束。

程序

```
n = 0
while n < 3:
    n += 1
```

查看结果 `n` → `3`

`continue` 流程控制

跳过最内层循环本轮剩余语句。for 接着取下一个元素，while 重新检查条件。while 中使用时，确认该分支也能推进循环条件。

程序

```
even = []
for x in [1, 2, 3, 4]:
    if x % 2:
        continue
    even.append(x)
```

查看结果 `even` → `[2, 4]`

`break` 流程控制

立即结束最内层循环，执行循环之后的代码。该循环若带有 else，会跳过 else。

程序

```
visited = []
for x in [1, 2, 3, 4]:
    if x == 3:
        break
    visited.append(x)
```

查看结果 `visited` → **`[1, 2]`**

`return` 表达式 流程控制

立即退出当前函数，结束其中正在执行的循环，并把表达式的值交给调用者。单写 return 或执行到函数末尾时，返回 None。

程序

```
def first_even(nums):
    for x in nums:
        if x % 2 == 0:
            return x
```

查看结果 `(first_even([1, 4, 6]), first_even([1, 3]))` → **`(4, None)`**

`for …: … else: …` / `while …: … else: …` 流程控制

else 与 for 或 while 对齐。for 耗尽元素、while 条件变假时执行 else，零次循环也算正常结束。continue 后若正常结束，仍执行 else。break、return 或异常使循环提前退出时，跳过 else。

程序

```
def contains_two(nums):
    for x in nums:
        if x == 2:
            break
    else:
        return False
    return True
```

查看结果 `(contains_two([1, 2]), contains_two([1, 3]), contains_two([]))` → **`(True, False, False)`**

程序

```
def finish(n):
    while n < 3:
        if n == 1:
            break
        n += 1
    else:
        return '条件变假'
    return '遇到 break'
```

查看结果 `(finish(0), finish(2), finish(3))` → **`('遇到 break', '条件变假', '条件变假')`**

`pass` 流程控制

空操作。需要占一个语句位置时可用，例如暂未实现的函数体。执行后继续运行后续语句。

程序

```
def placeholder():
    pass
```

查看结果 `placeholder() → None`

链表节点（判题平台提供）

普通链表题使用 ListNode。随机链表题使用带 random 的 Node。判题平台提供这两种定义，本地运行时可先复制对应定义。LRU 的双向 Node 见题内代码。

`ListNode(val=0, next=None)` 返回

返回单链表节点。val 是值，next 是下一个节点或 None。

程序

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
head = ListNode(1, ListNode(2))
```

查看结果 `(head.val, head.next.val, head.next.next) → (1, 2, None)`

`Node(x, next=None, random=None)` 返回

返回带随机指针的节点。val 保存 x，next 和 random 都可以是节点或 None。

程序

```
class Node:
    def __init__(self, x, next=None, random=None):
        self.val = x
        self.next = next
        self.random = random
head = Node(7)
head.random = head
```

查看结果 `(head.val, head.next, head.random is head) → (7, None, True)`

二叉树节点（判题平台提供）

LeetCode 的二叉树题已提供 TreeNode。下面是本地运行示例时可用的最小定义。

TreeNode(val=0, left=None, right=None) 返回

返回一个树节点，val 是节点值，left 和 right 是子节点或 None。

程序

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
root = TreeNode(1, TreeNode(2))
```

查看结果 (root.val, root.left.val, root.right) → **(1, 2, None)**